Carga completa del modelo estrella · Grano de Oro

5 transformaciones, 1 job

Generado el 21 de octubre de 2026, 16:11

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Documentos: job_carga_dw, 01_dim_tienda, 02_dim_producto, 03_dim_fecha, 04_hechos_ventas y 05_kpi_ventas

CONTENIDO

1. Documentos del informe

2. job_carga_dw

- 1.1 Resumen
- 1.2 Flujo del job
- 1.3 Métricas de ejecución
- 1.4 Validación
- 1.5 Conexiones y parámetros

3. 01_dim_tienda

- 2.1 Resumen
- 2.2 Diagrama de flujo
- 2.3 Linaje de campos
- 2.4 Diccionario de datos
- 2.5 Modelo destino
- 2.6 Validación
- 2.7 Conexiones y parámetros

4. 02_dim_producto

- 3.1 Resumen
- 3.2 Diagrama de flujo
- 3.3 Linaje de campos
- 3.4 Diccionario de datos
- 3.5 Modelo destino
- 3.6 Validación
- 3.7 Conexiones y parámetros

5. 03_dim_fecha

- 4.1 Resumen
- 4.2 Diagrama de flujo
- 4.3 Linaje de campos
- 4.4 Diccionario de datos
- 4.5 Modelo destino
- 4.6 Validación
- 4.7 Conexiones y parámetros

6. 04_hechos_ventas

- 5.1 Resumen
- 5.2 Diagrama de flujo
- 5.3 Linaje de campos
- 5.4 Diccionario de datos
- 5.5 Modelo destino
- 5.6 Métricas de ejecución
- 5.7 Validación
- 5.8 Conexiones y parámetros

7. 05_kpi_ventas

- 6.1 Resumen
- 6.2 Diagrama de flujo
- 6.3 Linaje de campos
- 6.4 Diccionario de datos
- 6.5 Modelo destino
- 6.6 Validación
- 6.7 Conexiones y parámetros

Documentos del informe

> Este informe reúne 6 documentos (5 transformaciones, 1 job). Cada uno tiene su capítulo con sus propias secciones.

N.º	Documento	Tipo	Archivo	Secciones
1	job_carga_dw	Job	job_carga_dw.kjb	6 completas
2	01_dim_tienda	Transformación	01_dim_tienda.ktr	8 completas
3	02_dim_producto	Transformación	02_dim_producto.ktr	8 completas
4	03_dim_fecha	Transformación	03_dim_fecha.ktr	8 completas
5	04_hechos_ventas	Transformación	04_hechos_ventas.ktr	9 completas
6	05_kpi_ventas	Transformación	05_kpi_ventas.ktr	8 completas

En pocas palabras

- **job_carga_dw.** Ejecuta 5 transformaciones (01_dim_tienda.ktr, 02_dim_producto.ktr, 03_dim_fecha.ktr, 04_hechos_ventas.ktr, 05_kpi_ventas.ktr), maneja archivos, ejecuta SQL y escribe en el log. 12 entradas y 18 hops.
- **01_dim_tienda.** Lee 1 archivo (tiendas.json), calcula, y escribe en 1 tabla (dim_tienda). 5 steps y 4 hops.
- **02_dim_producto.** Calcula y ordena, y escribe en 1 tabla (dim_producto). 5 steps y 4 hops.
- **03_dim_fecha.** Genera filas, calcula y selecciona campos, y escribe en 1 tabla (dim_fecha). 6 steps y 5 hops.
- **04_hechos_ventas.** Lee 1 archivo (ventas_2026_09.csv), calcula, selecciona campos, ordena, quita duplicados, busca valores, agrega constantes, filtra y separa las filas con error, escribe 1 archivo (rechazos_ventas.csv), y escribe en 1 tabla (hechos_ventas). 16 steps y 16 hops.
- **05_kpi_ventas.** Consulta 1 base (dw_grano_de_oro), ordena, agrupa y calcula, escribe 2 archivos (kpi_ticket_medio_tienda.csv, kpi_ventas_tienda_categoria.csv), y escribe en 2 tablas (kpi_ticket_medio_tienda, kpi_ventas_tienda_categoria). 11 steps y 10 hops.

JOB · JOB_CARGA_DW

1 job_carga_dw

Informe del job job_carga_dw

Job job_carga_dw.

Documento: job_carga_dw.kjb

Tipo: Job

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:20

Modificado: 2026-10-20 23:20

1.1 Resumen

> Ejecuta 5 transformaciones (01_dim_tienda.ktr, 02_dim_producto.ktr, 03_dim_fecha.ktr, 04_hechos_ventas.ktr, 05_kpi_ventas.ktr), maneja archivos, ejecuta SQL y escribe en el log. 12 entradas y 18 hops.

ENTRADAS DEL JOB	HOPS	TRANSFORMACIONES QUE EJECUTA	JOBS QUE EJECUTA	CONEXIONES
12	18	5 01_dim_tienda.ktr, 02_dim_producto.ktr, 03_dim_fecha.ktr, 04_hechos_ventas.ktr, 05_kpi_ventas.ktr	0	1

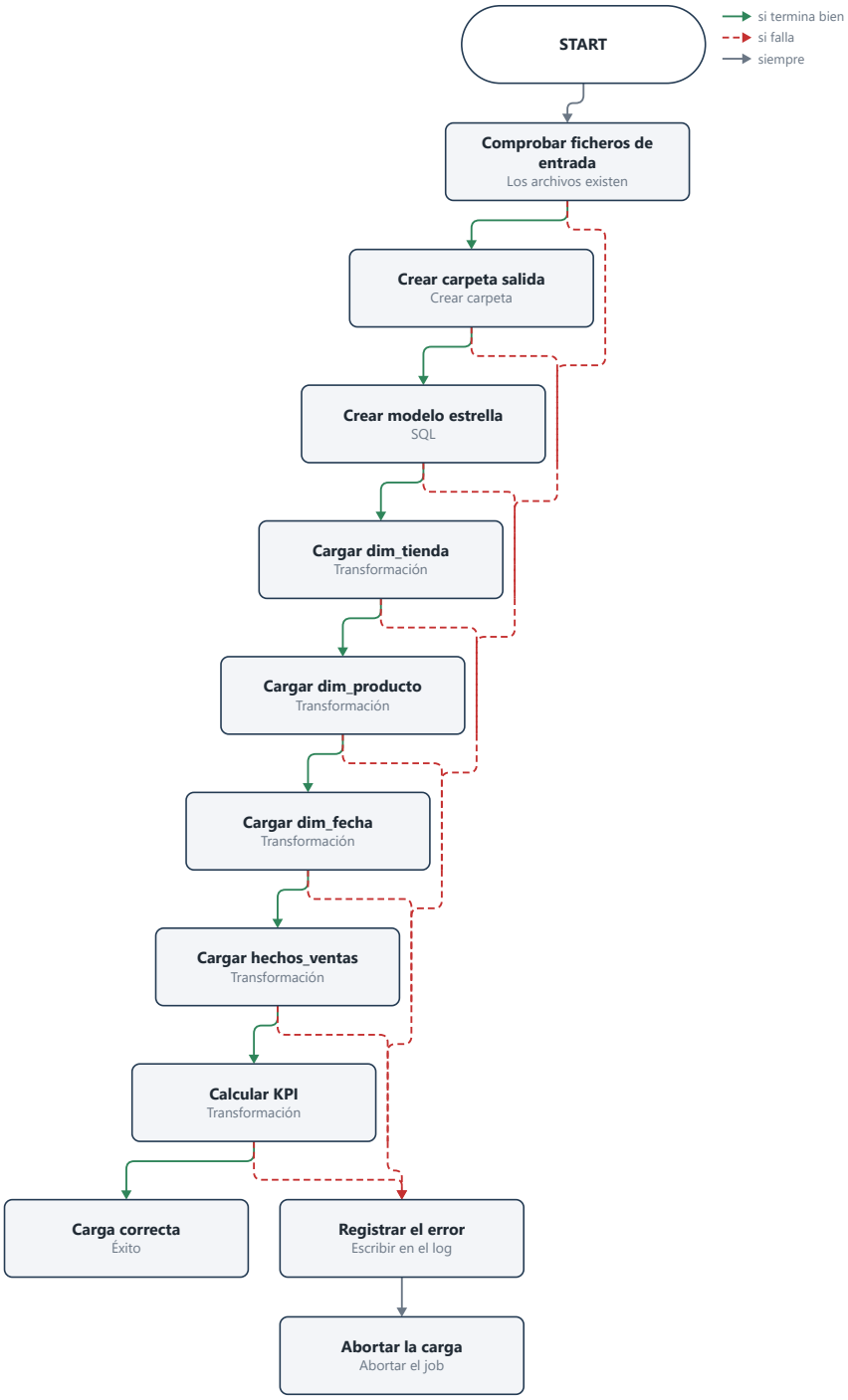
La última ejecución del job tardó 3,2 s y terminó bien; se ejecutaron 9 entradas. La entrada más lenta fue «Cargar dim_producto» (839 ms).

Lo más importante

- **Flujo del job:** 12 entradas y 18 hops. Ejecuta 5 transformaciones (01_dim_tienda.ktr, 02_dim_producto.ktr, 03_dim_fecha.ktr, 04_hechos_ventas.ktr, 05_kpi_ventas.ktr).
- **Validación:** 1 error y 0 advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

1.2 Flujo del job

> 12 entradas y 18 hops. Ejecuta 5 transformaciones (01_dim_tienda.ktr, 02_dim_producto.ktr, 03_dim_fecha.ktr, 04_hechos_ventas.ktr, 05_kpi_ventas.ktr).



Verde: si termina bien; rojo punteado: si falla; negro: siempre; las que terminan en x están deshabilitadas.

Entradas del job

#	Entrada	Tipo	Qué hace
1	START	Inicio (SPECIAL)	Inicio del job

#	Entrada	Tipo	Qué hace
2	Comprobar ficheros de entrada	Los archivos existen (FILES_EXIST)	Los archivos existen
3	Crear carpeta salida	Crear carpeta (CREATE_FOLDER)	Crear carpeta
4	Crear modelo estrella	SQL	Conexión: dw_grano_de_oro; SQL: DROP TABLE IF EXISTS hechos_ventas; DROP TABLE IF EXISTS kpi_ventas_tienda_categoria; DROP TABLE IF EXISTS kpi_ticket_medio_tienda; DROP TABLE IF EXISTS dim_...
5	Cargar dim_tienda	Transformación (TRANS)	Ejecuta la transformación: ${Internal.Entry.Current.Directory}/01_dim_tienda.ktr
6	Cargar dim_producto	Transformación (TRANS)	Ejecuta la transformación: ${Internal.Entry.Current.Directory}/02_dim_producto.ktr
7	Cargar dim_fecha	Transformación (TRANS)	Ejecuta la transformación: ${Internal.Entry.Current.Directory}/03_dim_fecha.ktr
8	Cargar hechos_ventas	Transformación (TRANS)	Ejecuta la transformación: ${Internal.Entry.Current.Directory}/04_hechos_ventas.ktr
9	Calcular KPI	Transformación (TRANS)	Ejecuta la transformación: ${Internal.Entry.Current.Directory}/05_kpi_ventas.ktr
10	Carga correcta	Éxito (SUCCESS)	Termina el job con éxito.
11	Registrar el error	Escribir en el log (WRITE_TO_LOG)	Escribe en el log: «La carga del almacén de Grano de Oro ha fallado: la entrada que ha fallado es la última que aparece como «Starting entry» antes de este mensaje.»
12	Abortar la carga	Abortar el job (ABORT)	Aborta el job: termina con error y el mensaje «Carga abortada: el almacén puede haber quedado a medias. Corrige el error y vuelve a lanzar el job.».

Qué ejecuta

Entrada	Tipo	Archivo	Existe
Cargar dim_tienda	Transformación	01_dim_tienda.ktr	sí
Cargar dim_producto	Transformación	02_dim_producto.ktr	sí
Cargar dim_fecha	Transformación	03_dim_fecha.ktr	sí
Cargar hechos_ventas	Transformación	04_hechos_ventas.ktr	sí
Calcular KPI	Transformación	05_kpi_ventas.ktr	sí

1.3 Métricas de ejecución

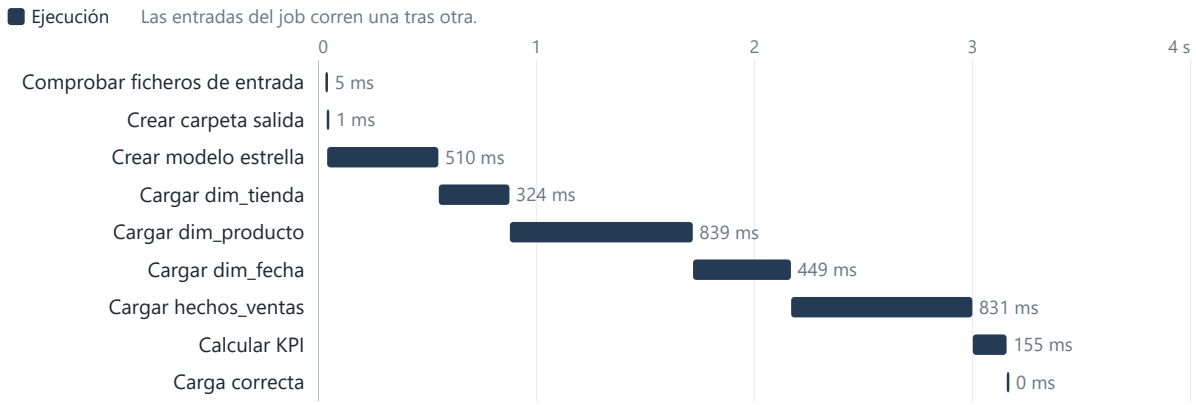
La última ejecución del job tardó 3,2 s y terminó bien; se ejecutaron 9 entradas. La entrada más lenta fue «Cargar dim_producto» (839 ms).

DURACIÓN	RESULTADO	ENTRADAS EJECUTADAS	ERRORES
3,2 s última ejecución en Spoon	Bien	9	0

ENTRADA MÁS LENTA
Cargar dim_producto
839 ms

Nota. Las filas de las transformaciones que ejecuta un job solo se conocen al medirlo (Medir...).

Línea de tiempo de las entradas



Las filas anidadas (↳) son del sub-job o de la sub-transformación de arriba.

Entradas del job

Entrada	Job	Inicio	Duración	Resultado	Llegó por
START	job_carga_dw	+26 ms	5 ms	Bien	el inicio del job
Comprobar ficheros de entrada	job_carga_dw	+32 ms	5 ms	Bien	un hop incondicional
Crear carpeta salida	job_carga_dw	+38 ms	1 ms	Bien	un hop «si termina bien»
Crear modelo estrella	job_carga_dw	+40 ms	510 ms	Bien	un hop «si termina bien»
Cargar dim_tienda	job_carga_dw	+552 ms	324 ms	Bien	un hop «si termina bien»
Cargar dim_producto	job_carga_dw	+878 ms	839 ms	Bien	un hop «si termina bien»
Cargar dim_fecha	job_carga_dw	+1,7 s	449 ms	Bien	un hop «si termina bien»
Cargar hechos_ventas	job_carga_dw	+2,2 s	831 ms	Bien	un hop «si termina bien»
Calcular KPI	job_carga_dw	+3,0 s	155 ms	Bien	un hop «si termina bien»
Carga correcta	job_carga_dw	+3,2 s	0 ms	Bien	un hop «si termina bien»

1.4 Validación

1 error y 0 advertencias.

ERRORES	ADVERTENCIAS
1	0

Step / entrada	Nivel	Mensaje
Crear carpeta salida	Error	filename cannot be null.

Los mensajes de Kettle salen en el idioma de Spoon; los chequeos de Pentaho Live, en el del informe.

1.5 Conexiones y parámetros

1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Crear modelo estrella

Nunca se incluyen contraseñas.

2 01_dim_tienda

Informe de la transformación 01_dim_tienda

Transformación 01_dim_tienda.

Documento: 01_dim_tienda.ktr

Tipo: Transformación

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:19

Modificado: 2026-10-20 23:19

2.1 Resumen

> Lee 1 archivo (tiendas.json), calcula, y escribe en 1 tabla (dim_tienda). 5 steps y 4 hops.

STEPS	HOPS	ARCHIVOS LEÍDOS	ARCHIVOS ESCRITOS
5	4	1 tiendas.json	0

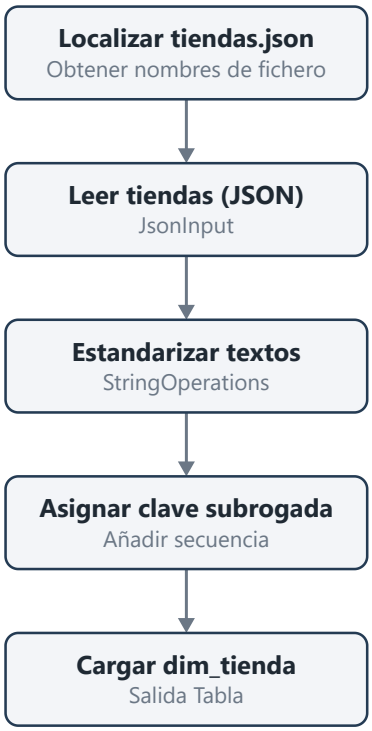
TABLAS ESCRITAS	CONEXIONES
1 dim_tienda	1

Lo más importante

- **Diagrama de flujo:** 5 steps y 4 hops. Sin hops de error.
- **Linaje de campos:** Traza 22 campos de «Cargar dim_tienda» hasta «Localizar tiendas.json» (tiendas.json). Operaciones: 4 cálculos y 9 campos creados en la transformación.
- **Diccionario de datos:** 1 salida final («Cargar dim_tienda») con 9 campos.
- **Modelo destino:** Escribe en 1 tabla (dim_tienda) por la conexión dw_grano_de_oro: 9 columnas. Las claves y los tipos salen de los steps, sin consultar la base.
- **Validación:** Sin errores ni advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

2.2 Diagrama de flujo

> 5 steps y 4 hops. Sin hops de error.



Las flechas llenas llevan filas; las punteadas rojas son hops de error; las que terminan en x están deshabilitadas.

Qué hace cada step

#	Step	Tipo	Qué hace
1	Localizar tiendas.json	Obtener nombres de fichero (GetFileNames)	Archivos: ../datos/tiendas.json
2	Leer tiendas (JSON)	JsonInput	JsonInput
3	Estandarizar textos	StringOperations	StringOperations
4	Asignar clave subrogada	Añadir secuencia (Sequence)	Obtener el siguiente valor de una secuencia
5	Cargar dim_tienda	Salida Tabla (TableOutput)	Tabla: dim_tienda; Conexión: dw_grano_de_oro

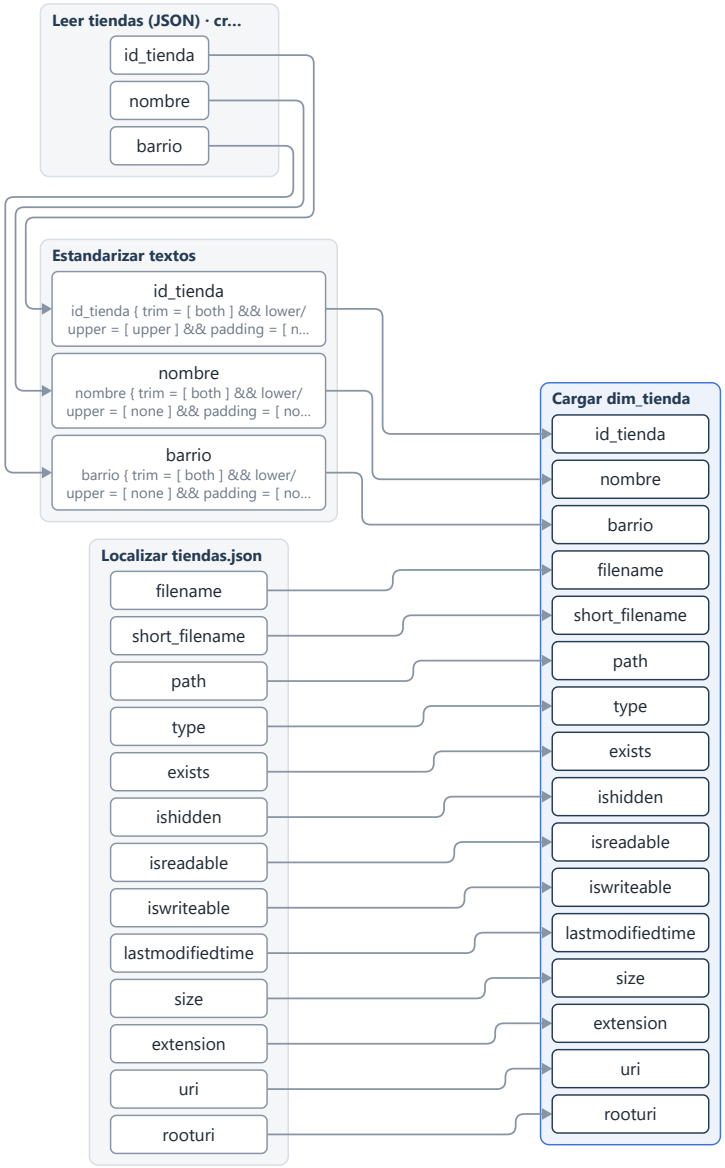
2.3 Linaje de campos

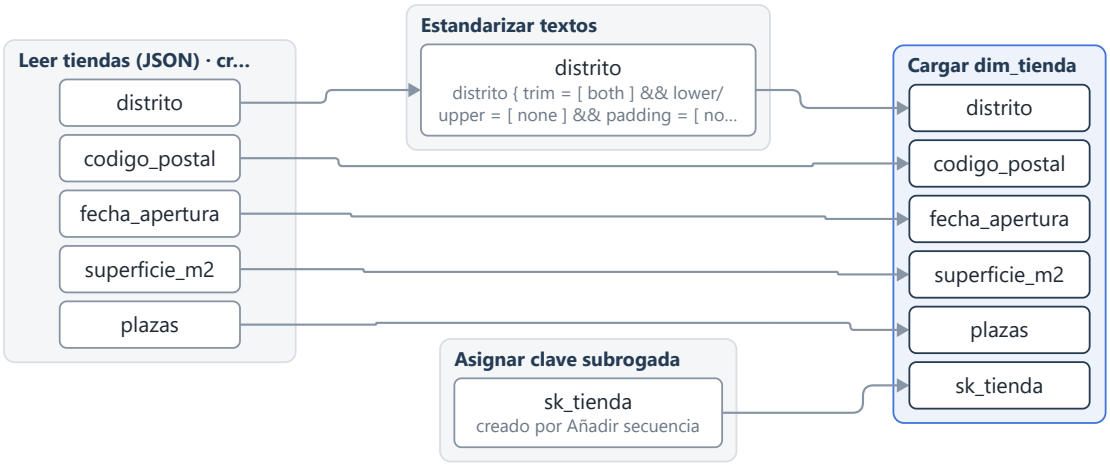
> Traza 22 campos de «Cargar dim_tienda» hasta «Localizar tiendas.json» (tiendas.json). Operaciones: 4 cálculos y 9 campos creados en la transformación.

Para qué sirve: muestra de dónde sale cada campo que escribe la transformación y qué le pasa por el camino. Sirve para saber qué se rompe si cambias una columna de origen (análisis de impacto), para encontrar en qué step se estropea un valor que sale mal y para documentar el proceso (auditorías y normativa).

Linaje hacia «Cargar dim_tienda»

Parte 1 de 2





Campos de «Cargar dim_tienda»

Campo	Tipo	Viene de	Transformaciones
filename	String	Localizar tiendas.json (tiendas.json): filename	sin cambios
short_filename	String	Localizar tiendas.json (tiendas.json): short_filename	sin cambios
path	String	Localizar tiendas.json (tiendas.json): path	sin cambios
type	String	Localizar tiendas.json (tiendas.json): type	sin cambios
exists	Boolean	Localizar tiendas.json (tiendas.json): exists	sin cambios
ishidden	Boolean	Localizar tiendas.json (tiendas.json): ishidden	sin cambios
isreadable	Boolean	Localizar tiendas.json (tiendas.json): isreadable	sin cambios
iswriteable	Boolean	Localizar tiendas.json (tiendas.json): iswriteable	sin cambios
lastmodifiedtime	Date	Localizar tiendas.json (tiendas.json): lastmodifiedtime	sin cambios
size	Integer	Localizar tiendas.json (tiendas.json): size	sin cambios
extension	String	Localizar tiendas.json (tiendas.json): extension	sin cambios
uri	String	Localizar tiendas.json (tiendas.json): uri	sin cambios
rooturi	String	Localizar tiendas.json (tiendas.json): rooturi	sin cambios

Campo	Tipo	Viene de	Transformaciones
id_tienda	String	Leer tiendas (JSON): id_tienda (creado)	creado por JsonInput en «Leer tiendas (JSON)» → id_tienda { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> id_tienda en «Estandarizar textos»
nombre	String	Leer tiendas (JSON): nombre (creado)	creado por JsonInput en «Leer tiendas (JSON)» → nombre { trim = [both] && lower/upper = [none] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> nombre en «Estandarizar textos»
barrio	String	Leer tiendas (JSON): barrio (creado)	creado por JsonInput en «Leer tiendas (JSON)» → barrio { trim = [both] && lower/upper = [none] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> barrio en «Estandarizar textos»
distrito	String	Leer tiendas (JSON): distrito (creado)	creado por JsonInput en «Leer tiendas (JSON)» → distrito { trim = [both] && lower/upper = [none] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> distrito en «Estandarizar textos»
codigo_postal	String	Leer tiendas (JSON): codigo_postal (creado)	creado por JsonInput en «Leer tiendas (JSON)»
fecha_apertura	String	Leer tiendas (JSON): fecha_apertura (creado)	creado por JsonInput en «Leer tiendas (JSON)»
superficie_m2	Integer	Leer tiendas (JSON): superficie_m2 (creado)	creado por JsonInput en «Leer tiendas (JSON)»
plazas	Integer	Leer tiendas (JSON): plazas (creado)	creado por JsonInput en «Leer tiendas (JSON)»
sk_tienda	Integer	Asignar clave subrogada: sk_tienda (creado)	creado por Añadir secuencia en «Asignar clave subrogada»

Motor: metaverse de Pentaho.

2.4 Diccionario de datos

> 1 salida final («Cargar dim_tienda») con 9 campos.

Salida final: Cargar dim_tienda

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
sk_tienda	Integer		0	####0;-####0	Asignar clave subrogada	
id_tienda	String				Leer tiendas (JSON)	
nombre	String				Leer tiendas (JSON)	
barrio	String				Leer tiendas (JSON)	
distrito	String				Leer tiendas (JSON)	
codigo_postal	String				Leer tiendas (JSON)	
fecha_apertura	String				Leer tiendas (JSON)	
superficie_m2	Integer		0		Leer tiendas (JSON)	
plazas	Integer		0		Leer tiendas (JSON)	

Nombres, orden, formato y largo como los escribe «Cargar dim_tienda»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Anexo: campos de los demás steps

Localizar tiendas.json

Campo	Tipo	Largo	Precisión	Origen
filename	String	500		Localizar tiendas.json
short_filename	String	500		Localizar tiendas.json
path	String	500		Localizar tiendas.json
type	String	500		Localizar tiendas.json
exists	Boolean			Localizar tiendas.json
ishidden	Boolean			Localizar tiendas.json
isreadable	Boolean			Localizar tiendas.json
iswriteable	Boolean			Localizar tiendas.json
lastmodifiedtime	Date			Localizar tiendas.json
size	Integer		0	Localizar tiendas.json
extension	String			Localizar tiendas.json
uri	String			Localizar tiendas.json
rooturi	String			Localizar tiendas.json

Leer tiendas (JSON)

Campo	Tipo	Largo	Precisión	Origen
filename	String	500		Localizar tiendas.json
short_filename	String	500		Localizar tiendas.json

Campo	Tipo	Largo	Precisión	Origen
path	String	500		Localizar tiendas.json
type	String	500		Localizar tiendas.json
exists	Boolean			Localizar tiendas.json
ishidden	Boolean			Localizar tiendas.json
isreadable	Boolean			Localizar tiendas.json
iswriteable	Boolean			Localizar tiendas.json
lastmodifiedtime	Date			Localizar tiendas.json
size	Integer		0	Localizar tiendas.json
extension	String			Localizar tiendas.json
uri	String			Localizar tiendas.json
rooturi	String			Localizar tiendas.json
id_tienda	String			Leer tiendas (JSON)
nombre	String			Leer tiendas (JSON)
barrio	String			Leer tiendas (JSON)
distrito	String			Leer tiendas (JSON)
codigo_postal	String			Leer tiendas (JSON)
fecha_apertura	String			Leer tiendas (JSON)
superficie_m2	Integer		0	Leer tiendas (JSON)
plazas	Integer		0	Leer tiendas (JSON)

Estandarizar textos

Campo	Tipo	Largo	Precisión	Origen
filename	String	500		Localizar tiendas.json
short_filename	String	500		Localizar tiendas.json
path	String	500		Localizar tiendas.json
type	String	500		Localizar tiendas.json
exists	Boolean			Localizar tiendas.json
ishidden	Boolean			Localizar tiendas.json
isreadable	Boolean			Localizar tiendas.json
iswriteable	Boolean			Localizar tiendas.json
lastmodifiedtime	Date			Localizar tiendas.json
size	Integer		0	Localizar tiendas.json
extension	String			Localizar tiendas.json
uri	String			Localizar tiendas.json
rooturi	String			Localizar tiendas.json
id_tienda	String			Leer tiendas (JSON)
nombre	String			Leer tiendas (JSON)

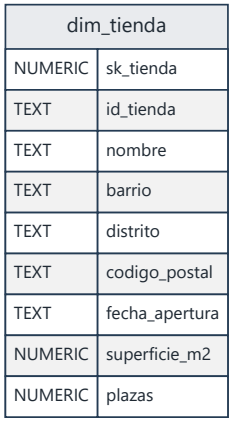
Campo	Tipo	Largo	Precisión	Origen
barrio	String			Leer tiendas (JSON)
distrito	String			Leer tiendas (JSON)
codigo_postal	String			Leer tiendas (JSON)
fecha_apertura	String			Leer tiendas (JSON)
superficie_m2	Integer		0	Leer tiendas (JSON)
plazas	Integer		0	Leer tiendas (JSON)

Asignar clave subrogada

Campo	Tipo	Largo	Precisión	Origen
filename	String	500		Localizar tiendas.json
short_filename	String	500		Localizar tiendas.json
path	String	500		Localizar tiendas.json
type	String	500		Localizar tiendas.json
exists	Boolean			Localizar tiendas.json
ishidden	Boolean			Localizar tiendas.json
isreadable	Boolean			Localizar tiendas.json
iswriteable	Boolean			Localizar tiendas.json
lastmodifiedtime	Date			Localizar tiendas.json
size	Integer		0	Localizar tiendas.json
extension	String			Localizar tiendas.json
uri	String			Localizar tiendas.json
rooturi	String			Localizar tiendas.json
id_tienda	String			Leer tiendas (JSON)
nombre	String			Leer tiendas (JSON)
barrio	String			Leer tiendas (JSON)
distrito	String			Leer tiendas (JSON)
codigo_postal	String			Leer tiendas (JSON)
fecha_apertura	String			Leer tiendas (JSON)
superficie_m2	Integer		0	Leer tiendas (JSON)
plazas	Integer		0	Leer tiendas (JSON)
sk_tienda	Integer		0	Asignar clave subrogada

2.5 Modelo destino

Escribe en 1 tabla (dim_tienda) por la conexión dw_grano_de_oro: 9 columnas. Las claves y los tipos salen de los steps, sin consultar la base.



dw_grano_de_oro › dim_tienda — Cargar dim_tienda (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
sk_tienda	sk_tienda	Integer	NUMERIC	
id_tienda	id_tienda	String	TEXT	
nombre	nombre	String	TEXT	
barrio	barrio	String	TEXT	
distrito	distrito	String	TEXT	
codigo_postal	codigo_postal	String	TEXT	
fecha_apertura	fecha_apertura	String	TEXT	
superficie_m2	superficie_m2	Integer	NUMERIC	
plazas	plazas	Integer	NUMERIC	

Nota. No se consultó la base: no se muestran las claves primarias ni foráneas, y los tipos SQL son los que generaría Kettle. Con «Consultar la base» se leen.

2.6 Validación

Sin errores ni advertencias.

ERRORES	ADVERTENCIAS
0	0

2.7 Conexiones y parámetros

> 1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Cargar dim_tienda

Nunca se incluyen contraseñas.

3 02_dim_producto

Informe de la transformación 02_dim_producto

Transformación 02_dim_producto.

Documento: 02_dim_producto.ktr

Tipo: Transformación

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:19

Modificado: 2026-10-20 23:19

3.1 Resumen

> Calcula y ordena, y escribe en 1 tabla (dim_producto). 5 steps y 4 hops.

STEPS	HOPS	ARCHIVOS LEÍDOS	ARCHIVOS ESCRITOS
5	4	0	0

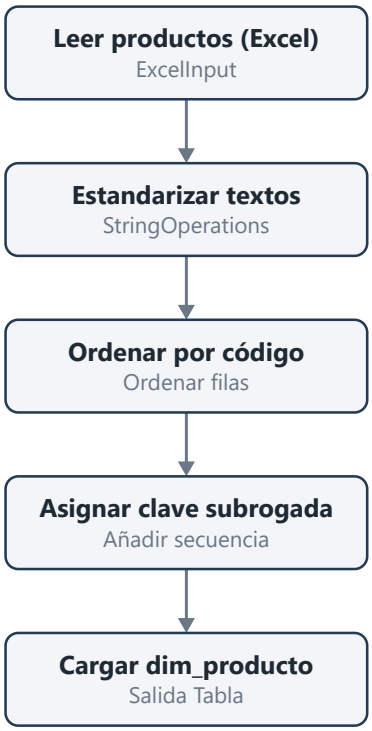
TABLAS ESCRITAS	CONEXIONES
1 dim_producto	1

Lo más importante

- **Diagrama de flujo:** 5 steps y 4 hops. Sin hops de error.
- **Linaje de campos:** Traza 5 campos de «Cargar dim_producto» hasta «Leer productos (Excel)» (ExcelInput). Operaciones: 3 cálculos y 1 campo creado en la transformación.
- **Diccionario de datos:** 1 salida final («Cargar dim_producto») con 5 campos.
- **Modelo destino:** Escribe en 1 tabla (dim_producto) por la conexión dw_grano_de_oro: 5 columnas. Las claves y los tipos salen de los steps, sin consultar la base.
- **Validación:** Sin errores ni advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

3.2 Diagrama de flujo

> 5 steps y 4 hops. Sin hops de error.



Las flechas llenas llevan filas; las punteadas rojas son hops de error; las que terminan en x están deshabilitadas.

Qué hace cada step

#	Step	Tipo	Qué hace
1	Leer productos (Excel)	ExcelInput	ExcelInput
2	Estandarizar textos	StringOperations	StringOperations
3	Ordenar por código	Ordenar filas (SortRows)	Ordena por: codigo_producto
4	Asignar clave subrogada	Añadir secuencia (Sequence)	Obtener el siguiente valor de una secuencia
5	Cargar dim_producto	Salida Tabla (TableOutput)	Tabla: dim_producto; Conexión: dw_grano_de_oro

3.3 Linaje de campos

> Traza 5 campos de «Cargar dim_producto» hasta «Leer productos (Excel)» (ExcelInput). Operaciones: 3 cálculos y 1 campo creado en la transformación.

Para qué sirve: muestra de dónde sale cada campo que escribe la transformación y qué le pasa por el camino. Sirve para saber qué se rompe si cambias una columna de origen (análisis de impacto), para encontrar en qué step se estropea un valor que sale mal y para documentar el proceso (auditorías y normativa).

Linaje hacia «Cargar dim_producto»



Campos de «Cargar dim_producto»

Campo	Tipo	Viene de	Transformaciones
codigo_producto	String	Leer productos (Excel) (ExcelInput): codigo_producto	codigo_producto { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> codigo_producto en «Estandarizar textos»
nombre	String	Leer productos (Excel) (ExcelInput): nombre	nombre { trim = [both] && lower/upper = [none] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> nombre en «Estandarizar textos»
categoria	String	Leer productos (Excel) (ExcelInput): categoria	categoria { trim = [both] && lower/upper = [none] && padding = [none,] && cap = [yes] && maskXML = [none] && digits = [none] && remove = [none] } -> categoria en «Estandarizar textos»
precio_venta	Number	Leer productos (Excel) (ExcelInput): precio_venta	sin cambios
sk_producto	Integer	Asignar clave subrogada: sk_producto (creado)	creado por Añadir secuencia en «Asignar clave subrogada»

Motor: metaverse de Pentaho.

3.4 Diccionario de datos

> 1 salida final («Cargar dim_producto») con 5 campos.

Salida final: Cargar dim_producto

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
sk_producto	Integer		0	####0;-####0	Asignar clave subrogada	
codigo_producto	String				Leer productos (Excel)	
nombre	String				Leer productos (Excel)	
categoria	String				Leer productos (Excel)	
precio_venta	Number				Leer productos (Excel)	

Nombres, orden, formato y largo como los escribe «Cargar dim_producto»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Anexo: campos de los demás steps

Leer productos (Excel)

Campo	Tipo	Largo	Precisión	Origen
codigo_producto	String			Leer productos (Excel)
nombre	String			Leer productos (Excel)
categoria	String			Leer productos (Excel)
precio_venta	Number			Leer productos (Excel)

Estandarizar textos

Campo	Tipo	Largo	Precisión	Origen
codigo_producto	String			Leer productos (Excel)
nombre	String			Leer productos (Excel)
categoria	String			Leer productos (Excel)
precio_venta	Number			Leer productos (Excel)

Ordenar por código

Campo	Tipo	Largo	Precisión	Origen
codigo_producto	String			Leer productos (Excel)
nombre	String			Leer productos (Excel)
categoria	String			Leer productos (Excel)
precio_venta	Number			Leer productos (Excel)

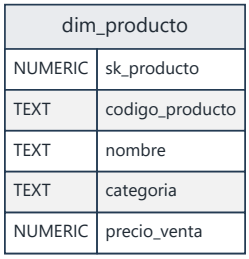
Asignar clave subrogada

Campo	Tipo	Largo	Precisión	Origen
codigo_producto	String			Leer productos (Excel)
nombre	String			Leer productos (Excel)

Campo	Tipo	Largo	Precisión	Origen
categoria	String			Leer productos (Excel)
precio_venta	Number			Leer productos (Excel)
sk_producto	Integer		0	Asignar clave subrogada

3.5 Modelo destino

Escribe en 1 tabla (dim_producto) por la conexión dw_grano_de_oro: 5 columnas. Las claves y los tipos salen de los steps, sin consultar la base.



dw_grano_de_oro › dim_producto — Cargar dim_producto (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
sk_producto	sk_producto	Integer	NUMERIC	
codigo_producto	codigo_producto	String	TEXT	
nombre	nombre	String	TEXT	
categoria	categoria	String	TEXT	
precio_venta	precio_venta	Number	NUMERIC	

Nota. No se consultó la base: no se muestran las claves primarias ni foráneas, y los tipos SQL son los que generaría Kettle. Con «Consultar la base» se leen.

3.6 Validación

Sin errores ni advertencias.

ERRORES	ADVERTENCIAS
0	0

3.7 Conexiones y parámetros

1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Cargar dim_producto

Nunca se incluyen contraseñas.

4 03_dim_fecha

Informe de la transformación 03_dim_fecha

Transformación 03_dim_fecha.

Documento: 03_dim_fecha.ktr

Tipo: Transformación

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:20

Modificado: 2026-10-20 23:20

4.1 Resumen

> Genera filas, calcula y selecciona campos, y escribe en 1 tabla (dim_fecha). 6 steps y 5 hops.

STEPS	HOPS	ARCHIVOS LEÍDOS	ARCHIVOS ESCRITOS
6	5	0	0

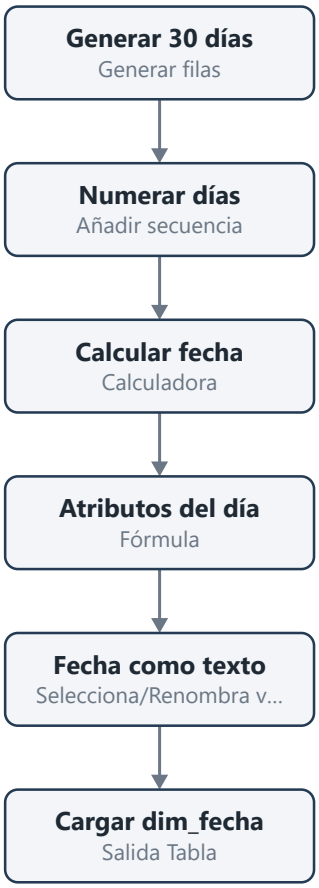
TABLAS ESCRITAS	CONEXIONES
1 dim_fecha	1

Lo más importante

- **Diagrama de flujo:** 6 steps y 5 hops. Sin hops de error.
- **Linaje de campos:** Traza 8 campos de «Cargar dim_fecha» hasta «Generar 30 días» (Generar filas). Operaciones: 8 cálculos, 1 conversión de tipo o formato y 5 campos creados en la transformación.
- **Diccionario de datos:** 1 salida final («Cargar dim_fecha») con 8 campos.
- **Modelo destino:** Escribe en 1 tabla (dim_fecha) por la conexión dw_grano_de_oro: 8 columnas. Las claves y los tipos salen de los steps, sin consultar la base.
- **Validación:** Sin errores ni advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

4.2 Diagrama de flujo

> 6 steps y 5 hops. Sin hops de error.



Las flechas llenas llevan filas; las punteadas rojas son hops de error; las que terminan en x están deshabilitadas.

Qué hace cada step

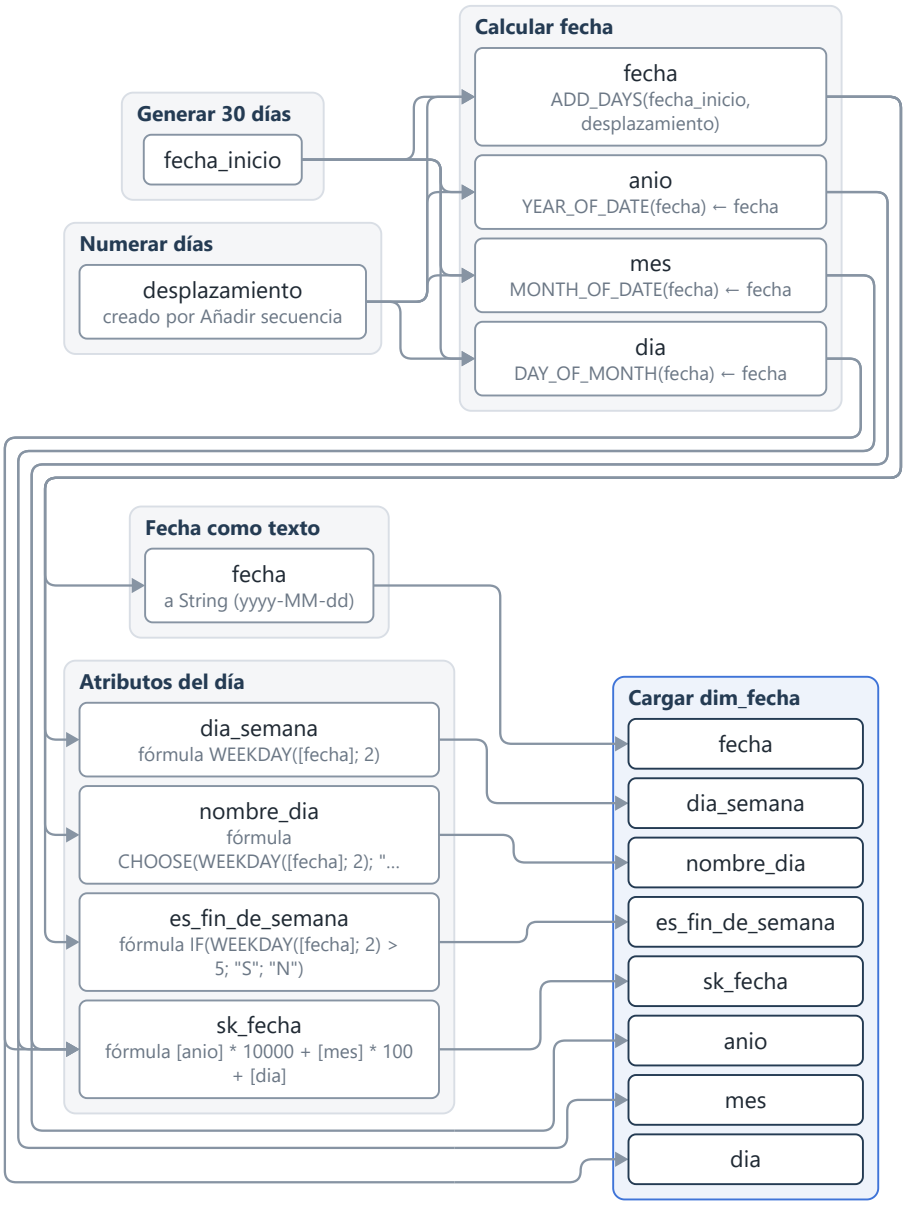
#	Step	Tipo	Qué hace
1	Generar 30 días	Generar filas (RowGenerator)	Generar filas vacías o iguales.
2	Numerar días	Añadir secuencia (Sequence)	Obtener el siguiente valor de una secuencia
3	Calcular fecha	Calculadora (Calculator)	Crear nuevos campos a partir de cálculos sencillos
4	Atributos del día	Fórmula (Formula)	Calcular una fórmula utilizando libformula de Pentaho
5	Fecha como texto	Selecciona/Renombra valores (SelectValues)	Seleccionar o eliminar campos de una fila. Opcionalmente, se puede establecer la meta-información del campo: tipo, longitud y precisión.
6	Cargar dim_fecha	Salida Tabla (TableOutput)	Tabla: dim_fecha; Conexión: dw_grano_de_oro

4.3 Linaje de campos

> Traza 8 campos de «Cargar dim_fecha» hasta «Generar 30 días» (Generar filas). Operaciones: 8 cálculos, 1 conversión de tipo o formato y 5 campos creados en la transformación.

Para qué sirve: muestra de dónde sale cada campo que escribe la transformación y qué le pasa por el camino. Sirve para saber qué se rompe si cambias una columna de origen (análisis de impacto), para encontrar en qué step se estropea un valor que sale mal y para documentar el proceso (auditorías y normativa).

Linaje hacia «Cargar dim_fecha»



Campos de «Cargar dim_fecha»

Campo	Tipo	Viene de	Transformaciones
fecha	String	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado)	ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → a String (yyyy-MM-dd) en «Fecha como texto»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → a String (yyyy-MM-dd) en «Fecha como texto»
anio	Integer	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado)	YEAR_OF_DATE(fecha) en «Calcular fecha»; ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → YEAR_OF_DATE(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → YEAR_OF_DATE(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → YEAR_OF_DATE(fecha) en «Calcular fecha»
mes	Integer	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado)	ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → MONTH_OF_DATE(fecha) en «Calcular fecha»; MONTH_OF_DATE(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → MONTH_OF_DATE(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → MONTH_OF_DATE(fecha) en «Calcular fecha»
dia	Integer	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado)	DAY_OF_MONTH(fecha) en «Calcular fecha»; ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → DAY_OF_MONTH(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → DAY_OF_MONTH(fecha) en «Calcular fecha»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → DAY_OF_MONTH(fecha) en «Calcular fecha»
sk_fecha	Integer	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado); Atributos del día: sk_fecha (creado)	YEAR_OF_DATE(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; DAY_OF_MONTH(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; MONTH_OF_DATE(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → YEAR_OF_DATE(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → DAY_OF_MONTH(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → MONTH_OF_DATE(fecha) en «Calcular fecha» → fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»; fórmula [anio] * 10000 + [mes] * 100 + [dia] en «Atributos del día»
dia_semana	Integer	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado); Atributos del día: dia_semana (creado)	ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula WEEKDAY([fecha]; 2) en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula WEEKDAY([fecha]; 2) en «Atributos del día»; fórmula WEEKDAY([fecha]; 2) en «Atributos del día»

Campo	Tipo	Viene de	Transformaciones
nombre_dia	String	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado); Atributos del día: nombre_dia (creado)	ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula CHOOSE(WEEKDAY([fecha]; 2); "lunes"; "martes"; "miércoles"; "jueves"; "viernes"; "sábado"; "domingo") en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula CHOOSE(WEEKDAY([fecha]; 2); "lunes"; "martes"; "miércoles"; "jueves"; "viernes"; "sábado"; "domingo") en «Atributos del día»; fórmula CHOOSE(WEEKDAY([fecha]; 2); "lunes"; "martes"; "miércoles"; "jueves"; "viernes"; "sábado"; "domingo") en «Atributos del día»
es_fin_de_semana	String	Generar 30 días (Generar filas): fecha_inicio; Numerar días: desplazamiento (creado); Atributos del día: es_fin_de_semana (creado)	ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula IF(WEEKDAY([fecha]; 2) > 5; "S"; "N") en «Atributos del día»; creado por Añadir secuencia en «Numerar días» → ADD_DAYS(fecha_inicio, desplazamiento) en «Calcular fecha» → fórmula IF(WEEKDAY([fecha]; 2) > 5; "S"; "N") en «Atributos del día»; fórmula IF(WEEKDAY([fecha]; 2) > 5; "S"; "N") en «Atributos del día»

Motor: metaverse de Pentaho y motor propio de Pentaho Live.

4.4 Diccionario de datos

> 1 salida final («Cargar dim_fecha») con 8 campos.

Salida final: Cargar dim_fecha

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
sk_fecha	Integer		0	####0;-####0	Atributos del día	
fecha	String			yyyy-MM-dd	Fecha como texto	ADD_DAYS
anio	Integer		0		Calcular fecha	YEAR_OF_DATE
mes	Integer		0		Calcular fecha	MONTH_OF_DATE
dia	Integer		0		Calcular fecha	DAY_OF_MONTH
dia_semana	Integer		0	####0;-####0	Atributos del día	
nombre_dia	String				Atributos del día	
es_fin_de_semana	String				Atributos del día	

Nombres, orden, formato y largo como los escribe «Cargar dim_fecha»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Anexo: campos de los demás steps

Generar 30 días

Campo	Tipo	Largo	Precisión	Origen
fecha_inicio	Date			Generar 30 días

Numerar días

Campo	Tipo	Largo	Precisión	Origen
fecha_inicio	Date			Generar 30 días
desplazamiento	Integer		0	Numerar días

Calcular fecha

Campo	Tipo	Largo	Precisión	Origen
fecha_inicio	Date			Generar 30 días
desplazamiento	Integer		0	Numerar días
fecha	Date			Calcular fecha
anio	Integer		0	Calcular fecha
mes	Integer		0	Calcular fecha
dia	Integer		0	Calcular fecha

Atributos del día

Campo	Tipo	Largo	Precisión	Origen
fecha_inicio	Date			Generar 30 días
desplazamiento	Integer		0	Numerar días

Campo	Tipo	Largo	Precisión	Origen
fecha	Date			Calcular fecha
anio	Integer		0	Calcular fecha
mes	Integer		0	Calcular fecha
dia	Integer		0	Calcular fecha
sk_fecha	Integer		0	Atributos del día
dia_semana	Integer		0	Atributos del día
nombre_dia	String			Atributos del día
es_fin_de_semana	String			Atributos del día

Fecha como texto

Campo	Tipo	Largo	Precisión	Origen
fecha	String			Fecha como texto
anio	Integer		0	Calcular fecha
mes	Integer		0	Calcular fecha
dia	Integer		0	Calcular fecha
sk_fecha	Integer		0	Atributos del día
dia_semana	Integer		0	Atributos del día
nombre_dia	String			Atributos del día
es_fin_de_semana	String			Atributos del día

4.5 Modelo destino

Escribe en 1 tabla (dim_fecha) por la conexión dw_grano_de_oro: 8 columnas. Las claves y los tipos salen de los steps, sin consultar la base.

dim_fecha	
NUMERIC	sk_fecha
TEXT	fecha
NUMERIC	anio
NUMERIC	mes
NUMERIC	dia
NUMERIC	dia_semana
TEXT	nombre_dia
TEXT	es_fin_de_semana

dw_grano_de_oro › dim_fecha — Cargar dim_fecha (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
sk_fecha	sk_fecha	Integer	NUMERIC	
fecha	fecha	String	TEXT	
anio	anio	Integer	NUMERIC	
mes	mes	Integer	NUMERIC	
dia	dia	Integer	NUMERIC	
dia_semana	dia_semana	Integer	NUMERIC	
nombre_dia	nombre_dia	String	TEXT	
es_fin_de_semana	es_fin_de_semana	String	TEXT	

Nota. No se consultó la base: no se muestran las claves primarias ni foráneas, y los tipos SQL son los que generaría Kettle. Con «Consultar la base» se leen.

4.6 Validación

Sin errores ni advertencias.

ERRORES	ADVERTENCIAS
0	0

4.7 Conexiones y parámetros

> 1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Cargar dim_fecha

Nunca se incluyen contraseñas.

5 04_hechos_ventas

Informe de la transformación 04_hechos_ventas

Transformación 04_hechos_ventas.

Documento: 04_hechos_ventas.ktr

Tipo: Transformación

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:20

Modificado: 2026-10-20 23:20

5.1 Resumen

> Lee 1 archivo (ventas_2026_09.csv), calcula, selecciona campos, ordena, quita duplicados, busca valores, agrega constantes, filtra y separa las filas con error, escribe 1 archivo (rechazos_ventas.csv), y escribe en 1 tabla (hechos_ventas). 16 steps y 16 hops.

STEPS	HOPS	ARCHIVOS LEÍDOS	ARCHIVOS ESCRITOS
16	16	1 ventas_2026_09.csv	1 rechazos_ventas.csv

TABLAS ESCRITAS	CONEXIONES
1 hechos_ventas	1

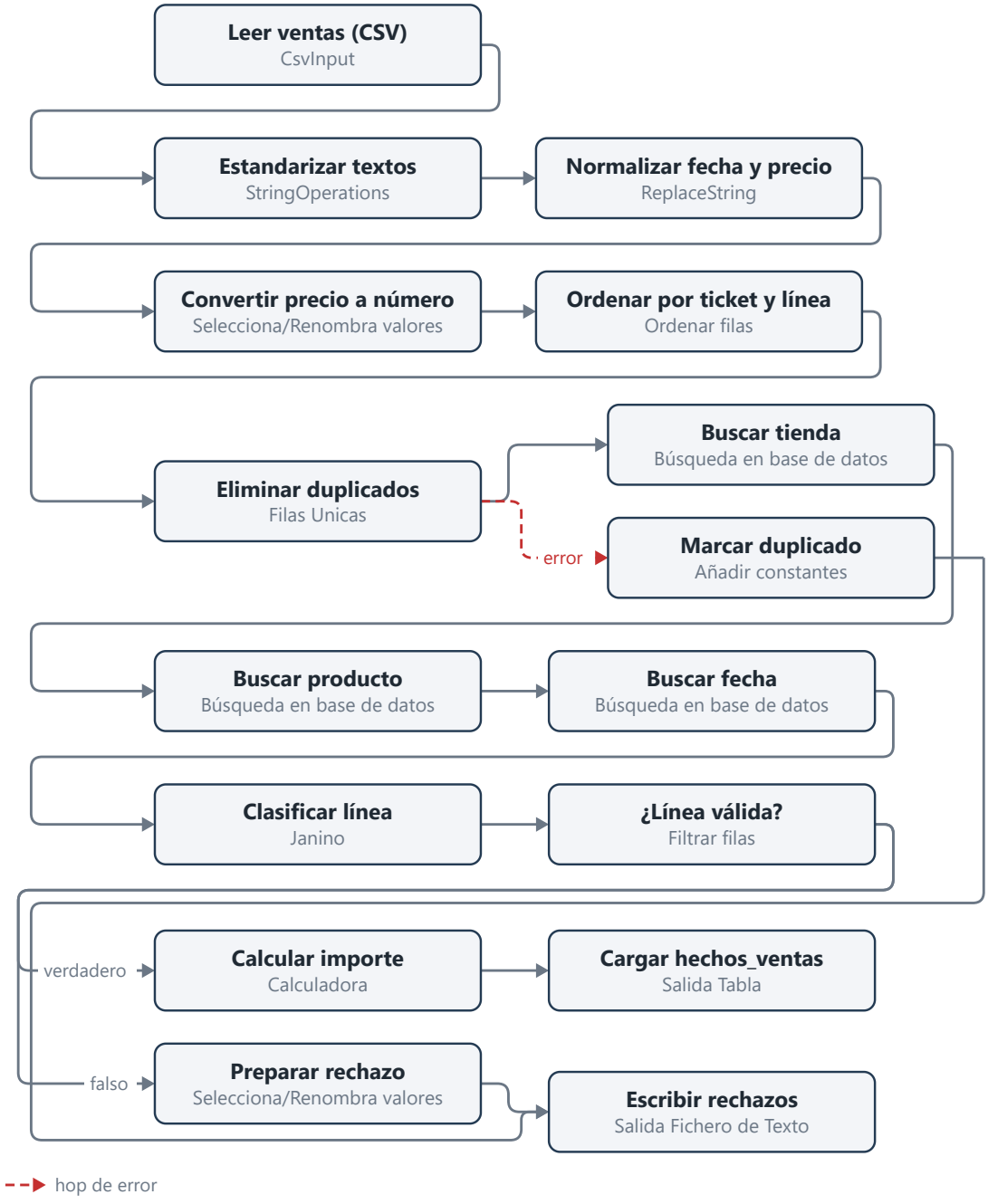
La última ejecución procesó 3.000 filas en 292 ms. El step más lento fue «Cargar hechos_ventas» (206 ms).

Lo más importante

- **Diagrama de flujo:** 16 steps y 16 hops. Tiene 1 hop de error.
- **Linaje de campos:** Traza 22 campos de 2 steps destino hasta «Leer ventas (CSV)» (ventas_2026_09.csv). Operaciones: 6 cálculos, 1 conversión de tipo o formato y 5 campos creados en la transformación.
- **Diccionario de datos:** 2 salidas finales con 18 campos en total.
- **Modelo destino:** Escribe en 1 tabla (hechos_ventas) por la conexión dw_grano_de_oro: 9 columnas. Las claves y los tipos salen de los steps, sin consultar la base.
- **Validación:** Sin errores ni advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

5.2 Diagrama de flujo

> 16 steps y 16 hops. Tiene 1 hop de error.



Las flechas llenas llevan filas; las punteadas rojas son hops de error; las que terminan en x están deshabilitadas.

Qué hace cada step

#	Step	Tipo	Qué hace
1	Leer ventas (CSV)	CsvInput	Lee: ${Internal.Entry.Current.Directory}/../datos/ventas_2026_09.csv
2	Estandarizar textos	StringOperations	StringOperations

#	Step	Tipo	Qué hace
3	Normalizar fecha y precio	ReplaceString	ReplaceString
4	Convertir precio a número	Selecciona/Renombra valores (SelectValues)	Seleccionar o eliminar campos de una fila. Opcionalmente, se puede establecer la meta-información del campo: tipo, longitud y precisión.
5	Ordenar por ticket y línea	Ordenar filas (SortRows)	Ordena por: id_ticket, linea
6	Eliminar duplicados	Filas Unicas (Unique)	Eliminar filas duplicadas y dejar solo ocurrencias únicas. Esto funciona solo con entradas ordenadas. Si la entrada no esta ordenada, solo se hace correctamente las entradas duplicadas consecutivas.
7	Buscar tienda	Búsqueda en base de datos (DBLookup)	Obtener valores de una base de datos utilizando campos
8	Buscar producto	Búsqueda en base de datos (DBLookup)	Obtener valores de una base de datos utilizando campos
9	Buscar fecha	Búsqueda en base de datos (DBLookup)	Obtener valores de una base de datos utilizando campos
10	Clasificar línea	Janino	Janino
11	Marcar duplicado	Añadir constantes (Constant)	Añadir uno o más constantes a las filas de entrada
12	¿Línea válida?	Filtrar filas (FilterRows)	Condición: motivo_rechazo IS NULL
13	Calcular importe	Calculadora (Calculator)	Crear nuevos campos a partir de cálculos sencillos
14	Cargar hechos_ventas	Salida Tabla (TableOutput)	Tabla: hechos_ventas; Conexión: dw_grano_de_oro
15	Preparar rechazo	Selecciona/Renombra valores (SelectValues)	Seleccionar o eliminar campos de una fila. Opcionalmente, se puede establecer la meta-información del campo: tipo, longitud y precisión.
16	Escribir rechazos	Salida Fichero de Texto (TextFileOutput)	Escribe: ${Internal.Entry.Current.Directory}/../salida/rechazos_ventas.csv

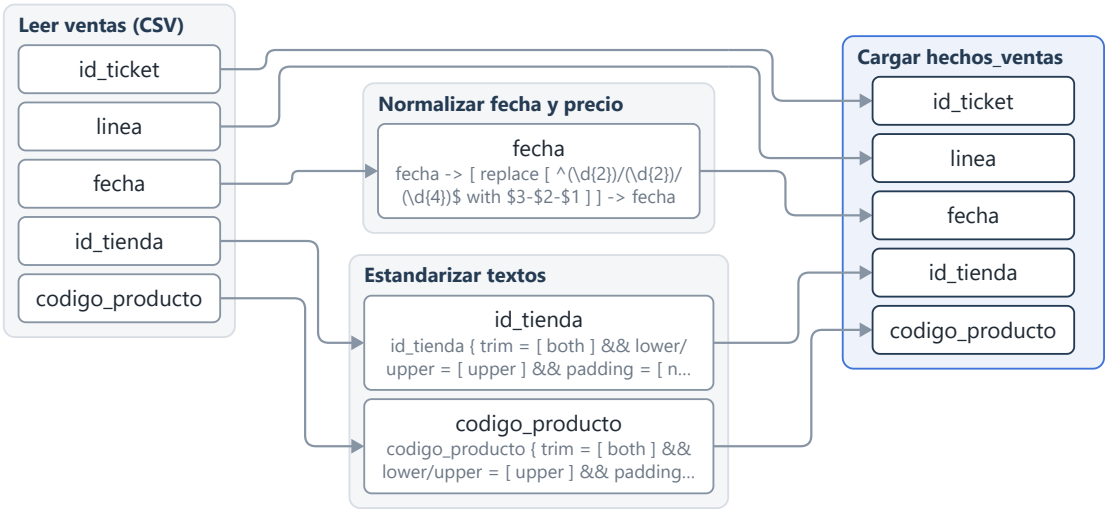
5.3 Linaje de campos

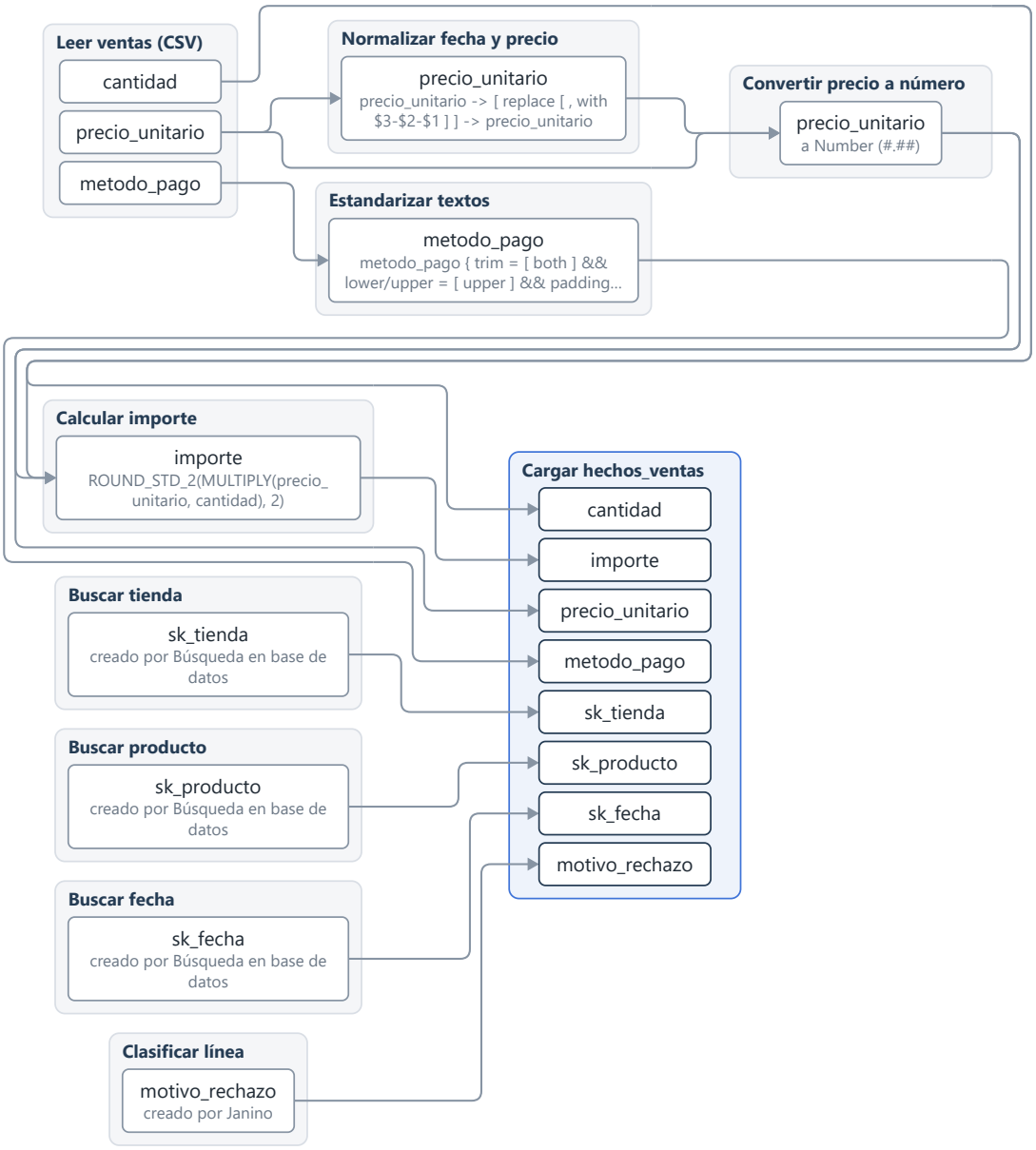
> Traza 22 campos de 2 steps destino hasta «Leer ventas (CSV)» (ventas_2026_09.csv). Operaciones: 6 cálculos, 1 conversión de tipo o formato y 5 campos creados en la transformación.

Para qué sirve: muestra de dónde sale cada campo que escribe la transformación y qué le pasa por el camino. Sirve para saber qué se rompe si cambias una columna de origen (análisis de impacto), para encontrar en qué step se estropea un valor que sale mal y para documentar el proceso (auditorías y normativa).

Linaje hacia «Cargar hechos_ventas»

Parte 1 de 2





Campos de «Cargar hechos_ventas»

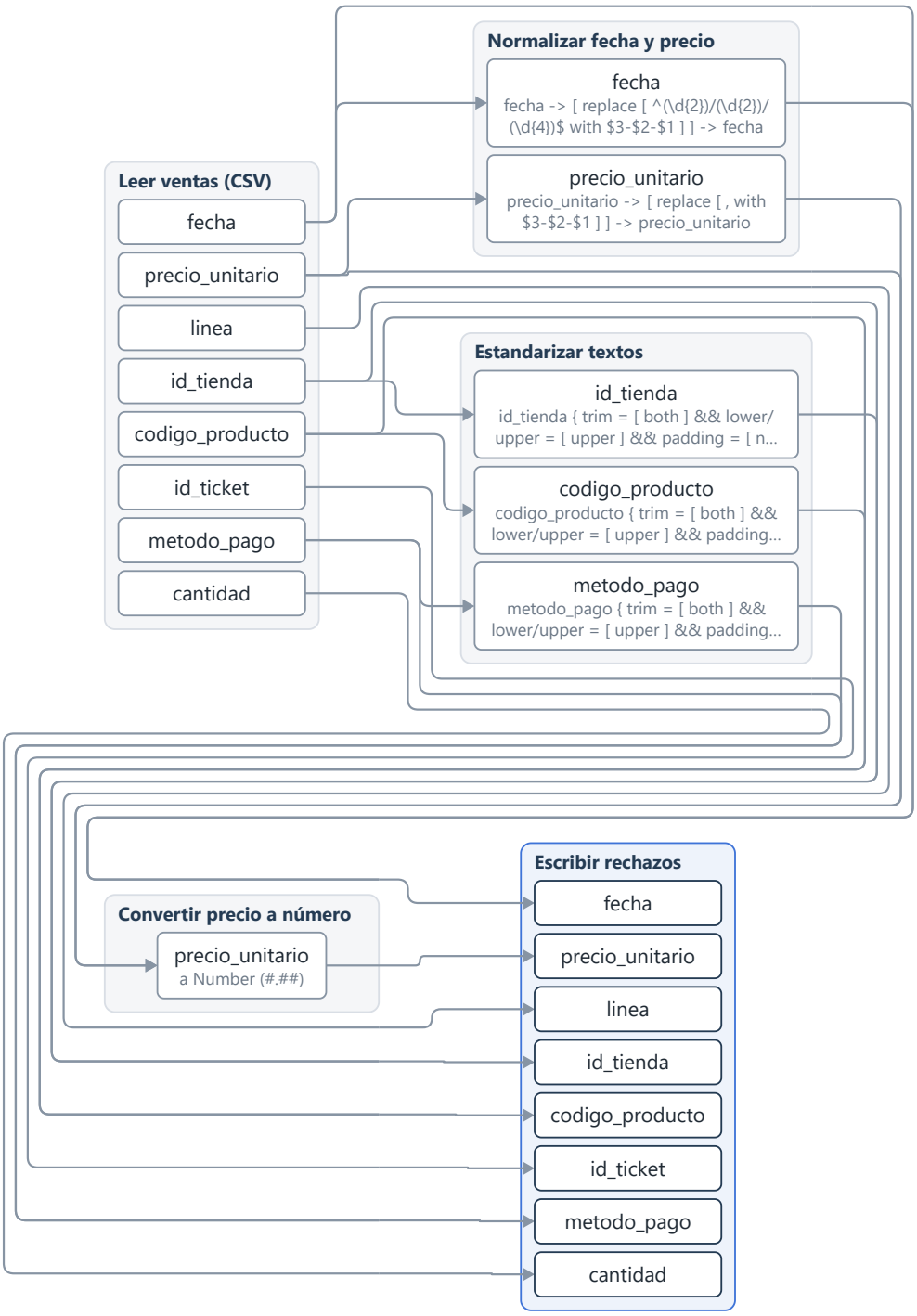
Campo	Tipo	Viene de	Transformaciones
id_ticket	String	Leer ventas (CSV) (ventas_2026_09.csv): id_ticket	sin cambios
linea	Integer	Leer ventas (CSV) (ventas_2026_09.csv): linea	sin cambios
fecha	String	Leer ventas (CSV) (ventas_2026_09.csv): fecha	fecha -> [replace [^(\d{2})/(\d{2})/(\d{4})$ with $3-$2-$1]] -> fecha en «Normalizar fecha y precio»
id_tienda	String	Leer ventas (CSV) (ventas_2026_09.csv): id_tienda	id_tienda { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> id_tienda en «Estandarizar textos»

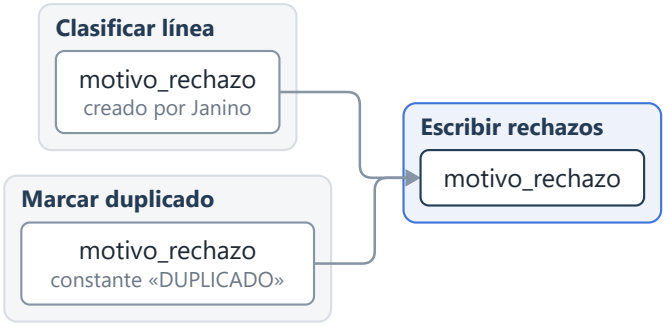
Campo	Tipo	Viene de	Transformaciones
codigo_producto	String	Leer ventas (CSV) (ventas_2026_09.csv): codigo_producto	codigo_producto { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> codigo_producto en «Estandarizar textos»
cantidad	Integer	Leer ventas (CSV) (ventas_2026_09.csv): cantidad	sin cambios
precio_unitario	Number	Leer ventas (CSV) (ventas_2026_09.csv): precio_unitario	precio_unitario -> [replace [, with $3-$2-$1]] -> precio_unitario en «Normalizar fecha y precio» → a Number (#.##) en «Convertir precio a número»
metodo_pago	String	Leer ventas (CSV) (ventas_2026_09.csv): metodo_pago	metodo_pago { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> metodo_pago en «Estandarizar textos»
sk_tienda	Integer	Buscar tienda: sk_tienda (creado)	creado por Búsqueda en base de datos en «Buscar tienda»
sk_producto	Integer	Buscar producto: sk_producto (creado)	creado por Búsqueda en base de datos en «Buscar producto»
sk_fecha	Integer	Buscar fecha: sk_fecha (creado)	creado por Búsqueda en base de datos en «Buscar fecha»
motivo_rechazo	String	Clasificar línea: motivo_rechazo (creado)	creado por Janino en «Clasificar línea»
importe	Number	Leer ventas (CSV) (ventas_2026_09.csv): cantidad, precio_unitario	ROUND_STD_2(MULTIPLY(precio_unitario, cantidad), 2) en «Calcular importe»; a Number (#.##) en «Convertir precio a número» → ROUND_STD_2(MULTIPLY(precio_unitario, cantidad), 2) en «Calcular importe»

Motor: metaverse de Pentaho y motor propio de Pentaho Live; metaverse no pudo trazar importe, así que esos campos los traza el motor propio.

Linaje hacia «Escribir rechazos»

Parte 1 de 2





Campos de «Escribir rechazos»

Campo	Tipo	Viene de	Transformaciones
id_ticket	String	Leer ventas (CSV) (ventas_2026_09.csv): id_ticket	sin cambios
linea	Integer	Leer ventas (CSV) (ventas_2026_09.csv): linea	sin cambios
fecha	String	Leer ventas (CSV) (ventas_2026_09.csv): fecha	fecha -> [replace [^(\d{2})/(\d{2})/(\d{4})$ with $3-$2-$1]] -> fecha en «Normalizar fecha y precio»
id_tienda	String	Leer ventas (CSV) (ventas_2026_09.csv): id_tienda	id_tienda { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> id_tienda en «Estandarizar textos»
codigo_producto	String	Leer ventas (CSV) (ventas_2026_09.csv): codigo_producto	codigo_producto { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> codigo_producto en «Estandarizar textos»
cantidad	Integer	Leer ventas (CSV) (ventas_2026_09.csv): cantidad	sin cambios
precio_unitario	Number	Leer ventas (CSV) (ventas_2026_09.csv): precio_unitario	precio_unitario -> [replace [, with $3-$2-$1]] -> precio_unitario en «Normalizar fecha y precio» → a Number (#.##) en «Convertir precio a número»; a Number (#.##) en «Convertir precio a número»
metodo_pago	String	Leer ventas (CSV) (ventas_2026_09.csv): metodo_pago	metodo_pago { trim = [both] && lower/upper = [upper] && padding = [none,] && cap = [no] && maskXML = [none] && digits = [none] && remove = [none] } -> metodo_pago en «Estandarizar textos»
motivo_rechazo	String	Clasificar línea: motivo_rechazo (creado); Marcar duplicado: motivo_rechazo (creado)	creado por Janino en «Clasificar línea»; constante «DUPLICADO» en «Marcar duplicado»

Motor: metaverse de Pentaho y motor propio de Pentaho Live.

5.4 Diccionario de datos

> 2 salidas finales con 18 campos en total.

Salida final: Cargar hechos_ventas

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
id_ticket	String				Leer ventas (CSV)	
linea	Integer		0	#	Leer ventas (CSV)	
sk_fecha	Integer	9	0	####0;-####0	Buscar fecha	sk_fecha
sk_tienda	Integer	9	0	####0;-####0	Buscar tienda	sk_tienda
sk_producto	Integer	9	0	####0;-####0	Buscar producto	sk_producto
cantidad	Integer		0	#	Leer ventas (CSV)	
precio_unitario	Number			#.##	Convertir precio a número	
importe	Number		2		Calcular importe	ROUND_STD_2
metodo_pago	String				Leer ventas (CSV)	

Nombres, orden, formato y largo como los escribe «Cargar hechos_ventas»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Salida final: Escribir rechazos

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
id_ticket	String				Leer ventas (CSV)	
linea	Integer		0	0	Leer ventas (CSV)	
fecha	String				Leer ventas (CSV)	
id_tienda	String				Leer ventas (CSV)	
codigo_producto	String				Leer ventas (CSV)	
cantidad	Integer		0	0	Leer ventas (CSV)	
precio_unitario	Number			0.00	Convertir precio a número	
metodo_pago	String				Leer ventas (CSV)	
motivo_rechazo	String				Marcar duplicado	

Nombres, orden, formato y largo como los escribe «Escribir rechazos»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Anexo: campos de los demás steps

Leer ventas (CSV)

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)

Campo	Tipo	Largo	Precisión	Origen
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	String			Leer ventas (CSV)
metodo_pago	String			Leer ventas (CSV)

Estandarizar textos

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	String			Leer ventas (CSV)
metodo_pago	String			Leer ventas (CSV)

Normalizar fecha y precio

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	String			Leer ventas (CSV)
metodo_pago	String			Leer ventas (CSV)

Convertir precio a número

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)

Ordenar por ticket y línea

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)

Eliminar duplicados

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)

Buscar tienda

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda

Buscar producto

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)

Campo	Tipo	Largo	Precisión	Origen
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda
sk_producto	Integer	9	0	Buscar producto

Buscar fecha

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda
sk_producto	Integer	9	0	Buscar producto
sk_fecha	Integer	9	0	Buscar fecha

Clasificar línea

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda
sk_producto	Integer	9	0	Buscar producto
sk_fecha	Integer	9	0	Buscar fecha
motivo_rechazo	String			Clasificar línea

Marcar duplicado

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
motivo_rechazo	String			Marcar duplicado

¿Línea válida?

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda
sk_producto	Integer	9	0	Buscar producto
sk_fecha	Integer	9	0	Buscar fecha
motivo_rechazo	String			Clasificar línea

Calcular importe

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
sk_tienda	Integer	9	0	Buscar tienda

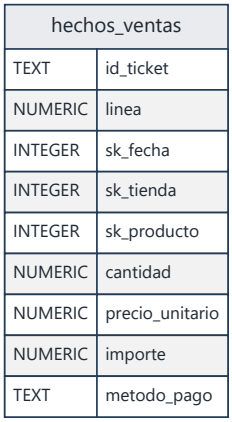
Campo	Tipo	Largo	Precisión	Origen
sk_producto	Integer	9	0	Buscar producto
sk_fecha	Integer	9	0	Buscar fecha
motivo_rechazo	String			Clasificar línea
importe	Number		2	Calcular importe

Preparar rechazo

Campo	Tipo	Largo	Precisión	Origen
id_ticket	String			Leer ventas (CSV)
linea	Integer		0	Leer ventas (CSV)
fecha	String			Leer ventas (CSV)
id_tienda	String			Leer ventas (CSV)
codigo_producto	String			Leer ventas (CSV)
cantidad	Integer		0	Leer ventas (CSV)
precio_unitario	Number			Convertir precio a número
metodo_pago	String			Leer ventas (CSV)
motivo_rechazo	String			Clasificar línea

5.5 Modelo destino

Escribe en 1 tabla (hechos_ventas) por la conexión dw_grano_de_oro: 9 columnas. Las claves y los tipos salen de los steps, sin consultar la base.



dw_grano_de_oro › hechos_ventas — Cargar hechos_ventas (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
id_ticket	id_ticket	String	TEXT	
linea	linea	Integer	NUMERIC	
sk_fecha	sk_fecha	Integer	INTEGER	
sk_tienda	sk_tienda	Integer	INTEGER	
sk_producto	sk_producto	Integer	INTEGER	
cantidad	cantidad	Integer	NUMERIC	
precio_unitario	precio_unitario	Number	NUMERIC	
importe	importe	Number	NUMERIC	
metodo_pago	metodo_pago	String	TEXT	

Nota. No se consultó la base: no se muestran las claves primarias ni foráneas, y los tipos SQL son los que generaría Kettle. Con «Consultar la base» se leen.

5.6 Métricas de ejecución

> La última ejecución procesó 3.000 filas en 292 ms. El step más lento fue «Cargar hechos_ventas» (206 ms).

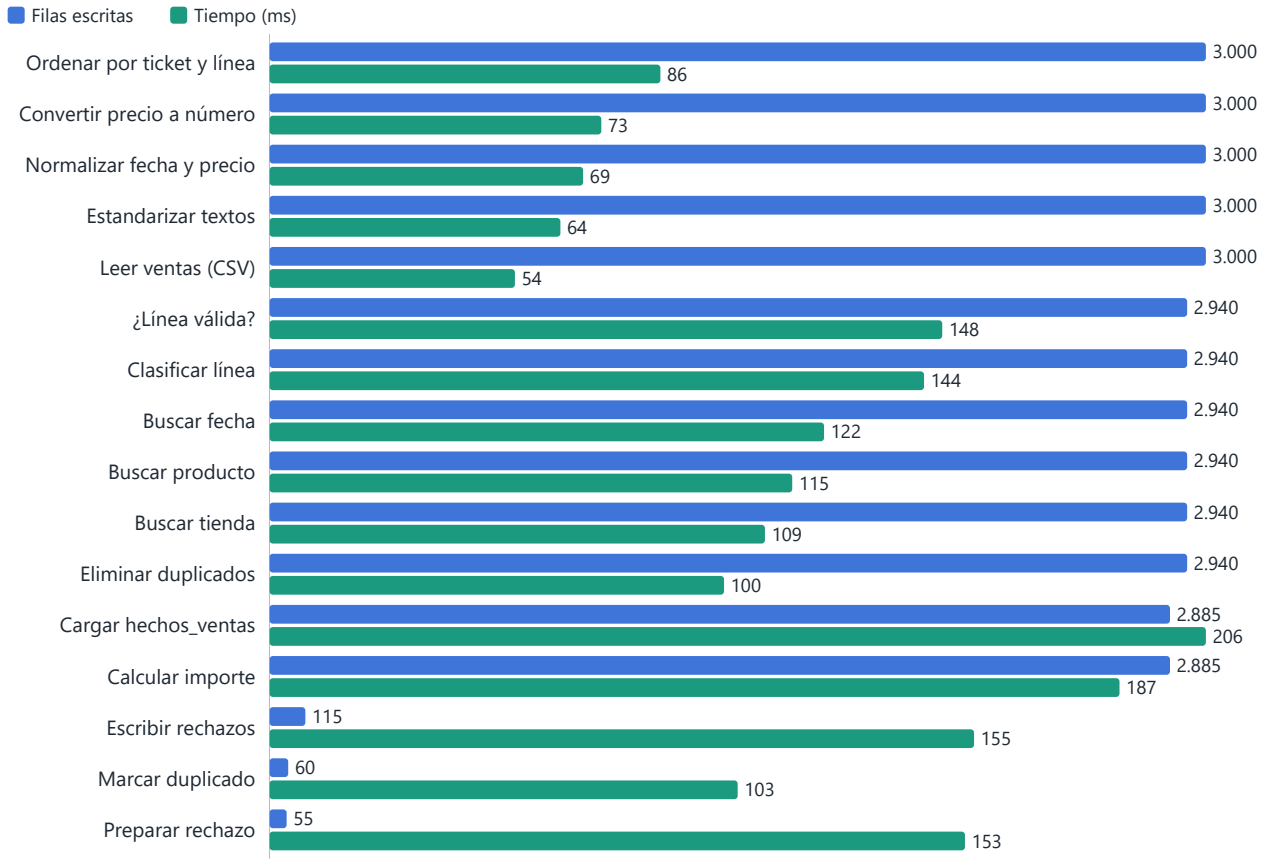
DURACIÓN	FILAS LEÍDAS	FILAS ESCRITAS	ERRORES
292 ms	3.000	3.000	0
última ejecución en Spoon	de las fuentes	por los steps de salida	

STEP MÁS LENTO

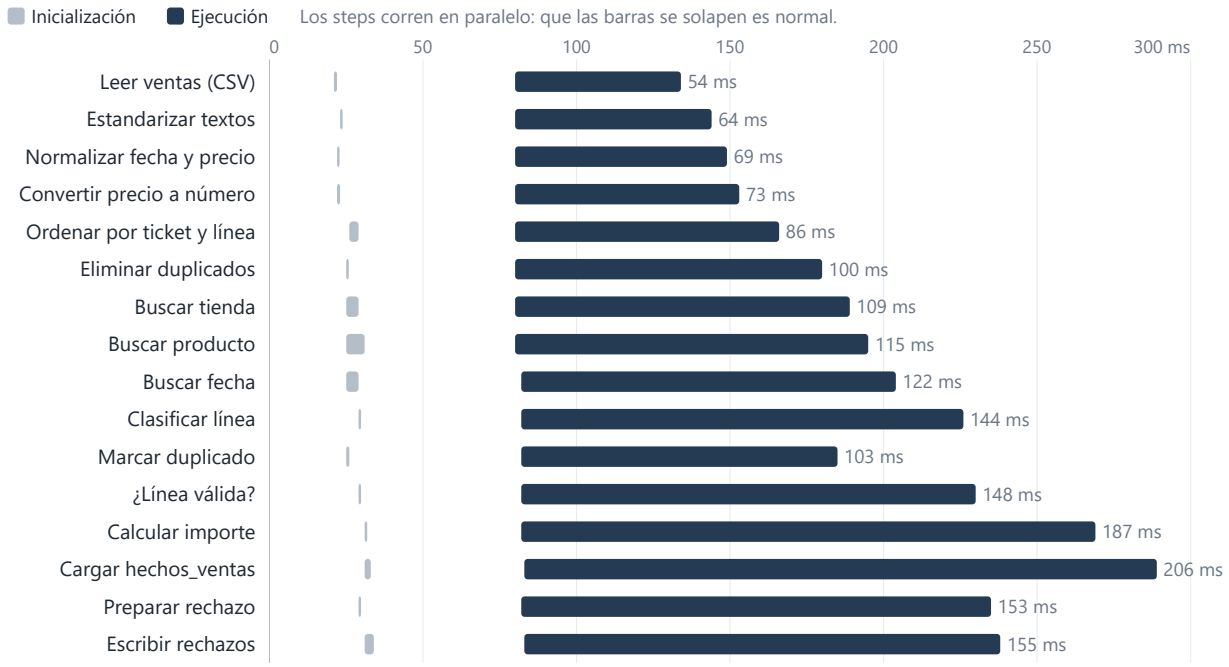
Cargar hechos_ventas

206 ms

Filas escritas y tiempo por step



Línea de tiempo de los steps



Métricas por step

Step	Leídas	Escritas	Entrada	Salida	Rechazadas	Errores	Estado	Tiempo	Filas/s
Leer ventas (CSV)	0	3.000	3.001	0	0	0	Terminado	54 ms	55.574
Estandarizar textos	3.000	3.000	0	0	0	0	Terminado	64 ms	46.875
Normalizar fecha y precio	3.000	3.000	0	0	0	0	Terminado	69 ms	43.478
Convertir precio a número	3.000	3.000	0	0	0	0	Terminado	73 ms	41.096
Ordenar por ticket y línea	3.000	3.000	0	0	0	0	Terminado	86 ms	34.884
Eliminar duplicados	3.000	2.940	0	0	60	0	Terminado	100 ms	30.000
Buscar tienda	2.940	2.940	4	0	0	0	Terminado	109 ms	26.972
Buscar producto	2.940	2.940	28	0	0	0	Terminado	115 ms	25.565
Buscar fecha	2.940	2.940	30	0	0	0	Terminado	122 ms	24.098
Clasificar línea	2.940	2.940	0	0	0	0	Terminado	144 ms	20.417
Marcar duplicado	60	60	0	0	0	0	Terminado	103 ms	583
¿Línea válida?	2.940	2.940	0	0	0	0	Terminado	148 ms	19.865
Calcular importe	2.885	2.885	0	0	0	0	Terminado	187 ms	15.428
Cargar hechos_ventas	2.885	2.885	0	2.885	0	0	Terminado	206 ms	14.005
Preparar rechazo	55	55	0	0	0	0	Terminado	153 ms	359
Escribir rechazos	115	115	0	116	0	0	Terminado	155 ms	748

5.7 Validación

Sin errores ni advertencias.

ERRORES	ADVERTENCIAS
0	0

5.8 Conexiones y parámetros

1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Buscar fecha, Buscar producto, Buscar tienda, Cargar hechos_ventas

Nunca se incluyen contraseñas.

6 05_kpi_ventas

Informe de la transformación 05_kpi_ventas

Transformación 05_kpi_ventas.

Documento: 05_kpi_ventas.ktr

Tipo: Transformación

Autor: Iván Castaño

Institución: Universidad Europea de Madrid

Curso: Proyecto de Big Data 1 · Práctica de ejemplo

Creado: 2026-10-20 23:20

Modificado: 2026-10-20 23:20

6.1 Resumen

> Consulta 1 base (dw_grano_de_oro), ordena, agrupa y calcula, escribe 2 archivos (kpi_ticket_medio_tienda.csv, kpi_ventas_tienda_categoria.csv), y escribe en 2 tablas (kpi_ticket_medio_tienda, kpi_ventas_tienda_categoria). 11 steps y 10 hops.

STEPS	HOPS	ARCHIVOS LEÍDOS
11	10	0

ARCHIVOS ESCRITOS	TABLAS ESCRITAS
2 kpi_ticket_medio_tienda.csv, kpi_ventas_tienda_categoria.csv	2 kpi_ticket_medio_tienda, kpi_ventas_tienda_categoria

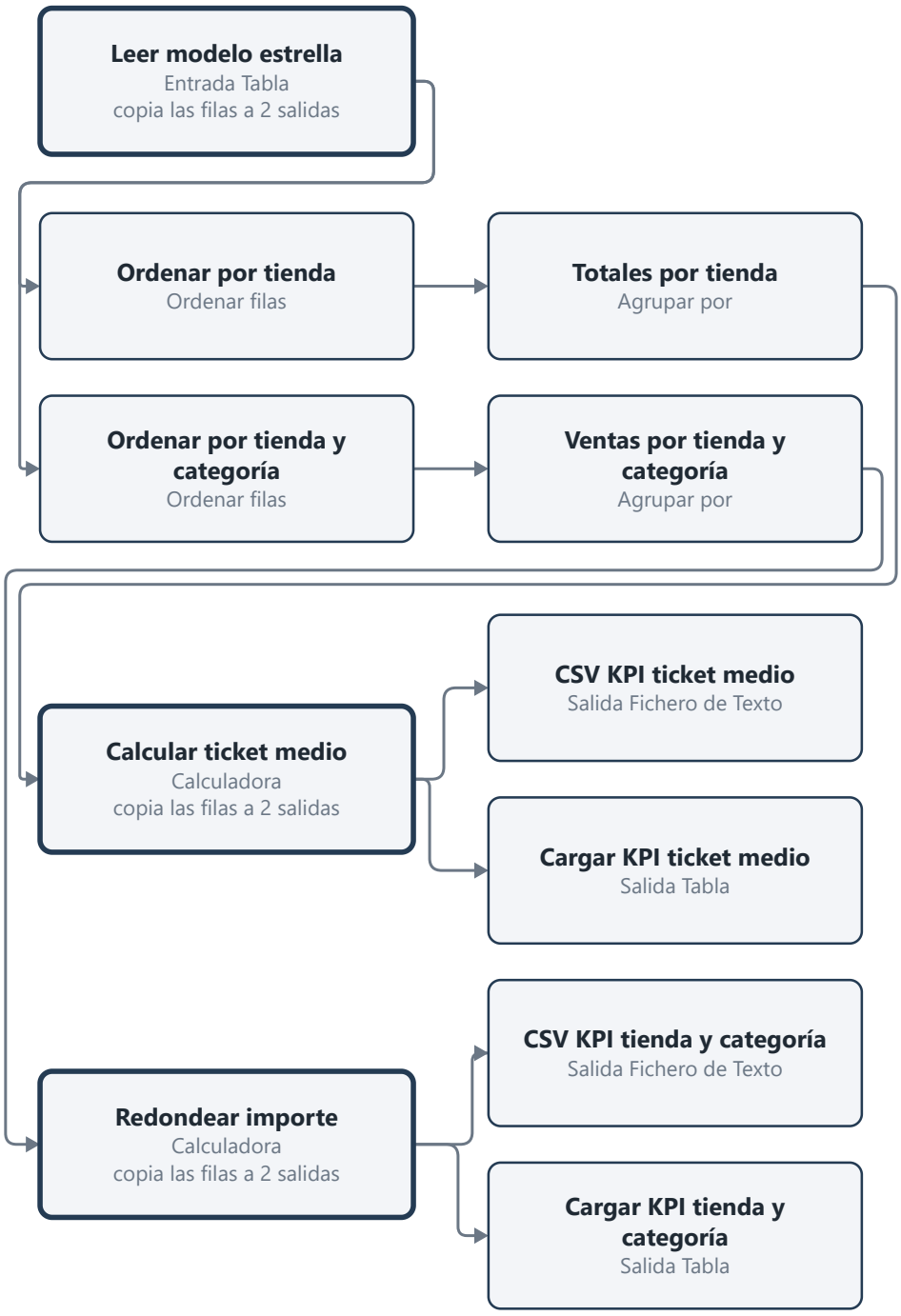
CONEXIONES
1

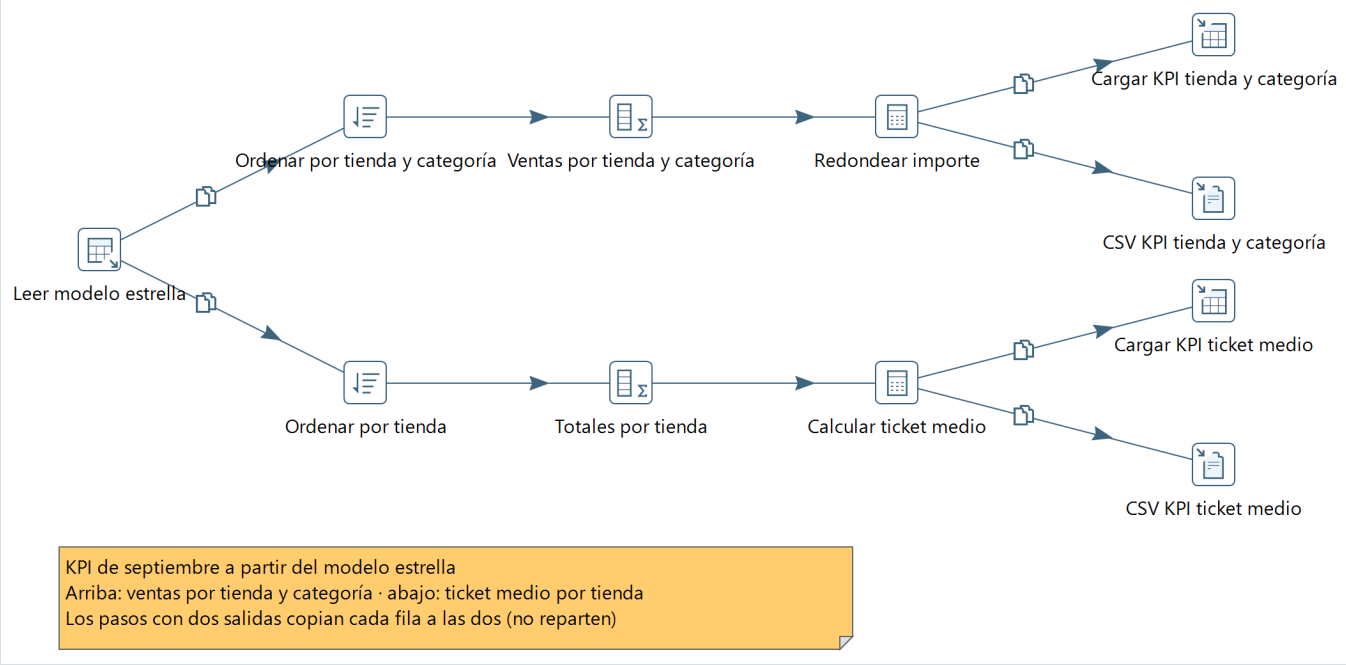
Lo más importante

- **Diagrama de flujo:** 11 steps y 10 hops. Sin hops de error.
- **Linaje de campos:** Traza 20 campos de 4 steps destino hasta «Leer modelo estrella» (consulta SQL a dw_grano_de_oro). Operaciones: 3 cálculos y 4 agregaciones.
- **Diccionario de datos:** 4 salidas finales con 16 campos en total.
- **Modelo destino:** Escribe en 2 tablas (kpi_ticket_medio_tienda, kpi_ventas_tienda_categoria): 8 columnas en total. Las claves y los tipos salen de los steps, sin consultar la base.
- **Validación:** Sin errores ni advertencias.
- **Conexiones y parámetros:** 1 conexión y 0 parámetros.

6.2 Diagrama de flujo

> 11 steps y 10 hops. Sin hops de error.





Las flechas llenas llevan filas; las punteadas rojas son hops de error; las que terminan en x están deshabilitadas.

Qué hace cada step

#	Step	Tipo	Qué hace
1	Leer modelo estrella	Entrada Tabla (TableInput)	Conexión: dw_grano_de_oro; SQL: SELECT t.nombre AS tienda, p.categoria, h.id_ticket, h.cantidad, h.importe FROM hechos_ventas h JOIN dim_tienda t ON t.sk_tienda = h.sk_tienda JOIN dim_produ...
2	Ordenar por tienda	Ordenar filas (SortRows)	Ordena por: tienda
3	Ordenar por tienda y categoría	Ordenar filas (SortRows)	Ordena por: tienda, categoria
4	Totales por tienda	Agrupar por (GroupBy)	Agrupa por: tienda
5	Calcular ticket medio	Calculadora (Calculator)	Crear nuevos campos a partir de cálculos sencillos
6	CSV KPI ticket medio	Salida Fichero de Texto (TextFileOutput)	Escribe: ${Internal.Entry.Current.Directory}/../salida/kpi_ticket_medio_tienda.csv
7	Cargar KPI ticket medio	Salida Tabla (TableOutput)	Tabla: kpi_ticket_medio_tienda; Conexión: dw_grano_de_oro
8	Ventas por tienda y categoría	Agrupar por (GroupBy)	Agrupa por: tienda, categoria
9	Redondear importe	Calculadora (Calculator)	Crear nuevos campos a partir de cálculos sencillos
10	CSV KPI tienda y categoría	Salida Fichero de Texto (TextFileOutput)	Escribe: ${Internal.Entry.Current.Directory}/../salida/kpi_ventas_tienda_categoria.csv

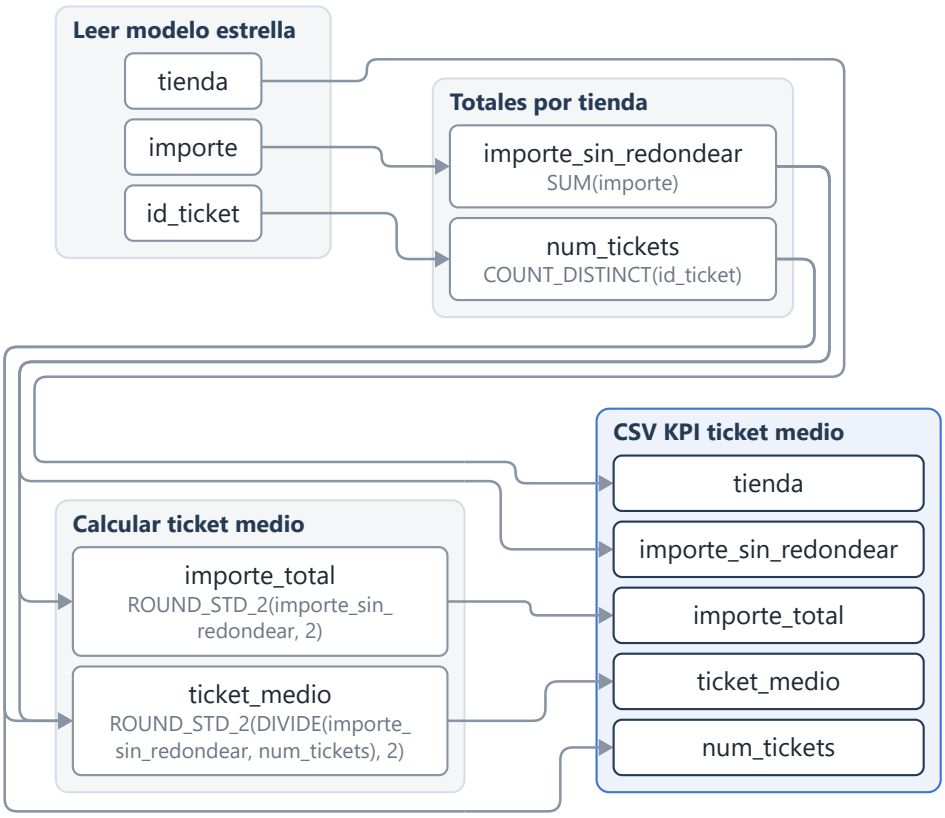
#	Step	Tipo	Qué hace
11	Cargar KPI tienda y categoría	Salida Tabla (TableOutput)	Tabla: kpi_ventas_tienda_categoria; Conexión: dw_grano_de_oro

6.3 Linaje de campos

> Traza 20 campos de 4 steps destino hasta «Leer modelo estrella» (consulta SQL a dw_grano_de_oro). Operaciones: 3 cálculos y 4 agregaciones.

Para qué sirve: muestra de dónde sale cada campo que escribe la transformación y qué le pasa por el camino. Sirve para saber qué se rompe si cambias una columna de origen (análisis de impacto), para encontrar en qué step se estropea un valor que sale mal y para documentar el proceso (auditorías y normativa).

Linaje hacia «CSV KPI ticket medio»



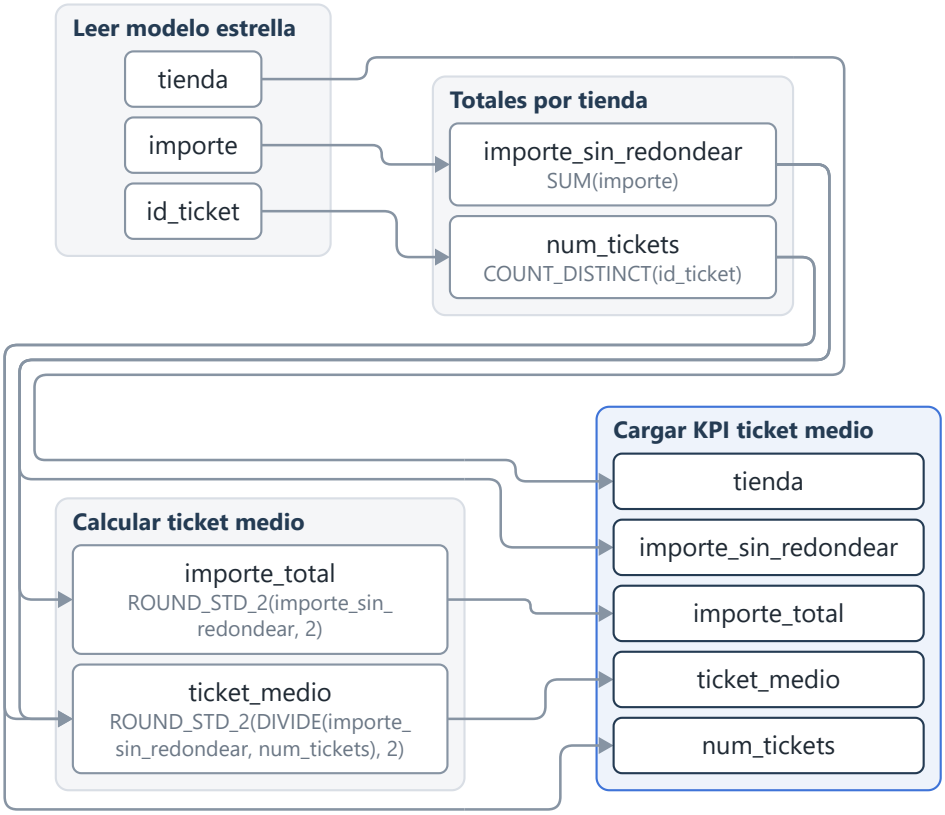
Campos de «CSV KPI ticket medio»

Campo	Tipo	Viene de	Transformaciones
tienda	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): tienda	sin cambios
importe_sin_redondear	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Totales por tienda»
num_tickets	Integer	Leer modelo estrella (consulta SQL a dw_grano_de_oro): id_ticket	COUNT_DISTINCT(id_ticket) en «Totales por tienda»

Campo	Tipo	Viene de	Transformaciones
importe_total	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Totales por tienda» → ROUND_STD_2(importe_sin_redondear, 2) en «Calcular ticket medio»
ticket_medio	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): id_ticket, importe	COUNT_DISTINCT(id_ticket) en «Totales por tienda» → ROUND_STD_2(DIVIDE(importe_sin_redondear, num_tickets), 2) en «Calcular ticket medio»; SUM(importe) en «Totales por tienda» → ROUND_STD_2(DIVIDE(importe_sin_redondear, num_tickets), 2) en «Calcular ticket medio»

Motor: metaverse de Pentaho y motor propio de Pentaho Live; metaverse no pudo trazar importe_total y ticket_medio, así que esos campos los traza el motor propio.

Linaje hacia «Cargar KPI ticket medio»



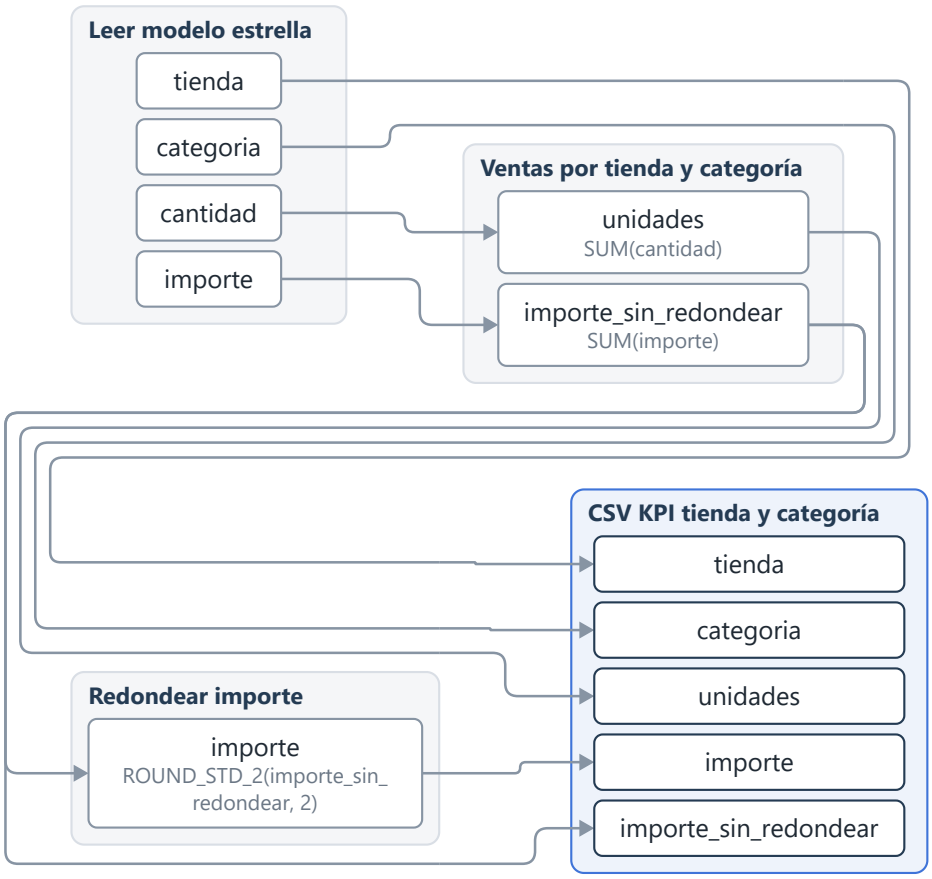
Campos de «Cargar KPI ticket medio»

Campo	Tipo	Viene de	Transformaciones
tienda	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): tienda	sin cambios
importe_sin_redondear	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Totales por tienda»

Campo	Tipo	Viene de	Transformaciones
num_tickets	Integer	Leer modelo estrella (consulta SQL a dw_grano_de_oro): id_ticket	COUNT_DISTINCT(id_ticket) en «Totales por tienda»
importe_total	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Totales por tienda» → ROUND_STD_2(importe_sin_redondear, 2) en «Calcular ticket medio»
ticket_medio	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): id_ticket, importe	COUNT_DISTINCT(id_ticket) en «Totales por tienda» → ROUND_STD_2(DIVIDE(importe_sin_redondear, num_tickets), 2) en «Calcular ticket medio»; SUM(importe) en «Totales por tienda» → ROUND_STD_2(DIVIDE(importe_sin_redondear, num_tickets), 2) en «Calcular ticket medio»

Motor: metaverse de Pentaho y motor propio de Pentaho Live; metaverse no pudo trazar importe_total y ticket_medio, así que esos campos los traza el motor propio.

Linaje hacia «CSV KPI tienda y categoría»



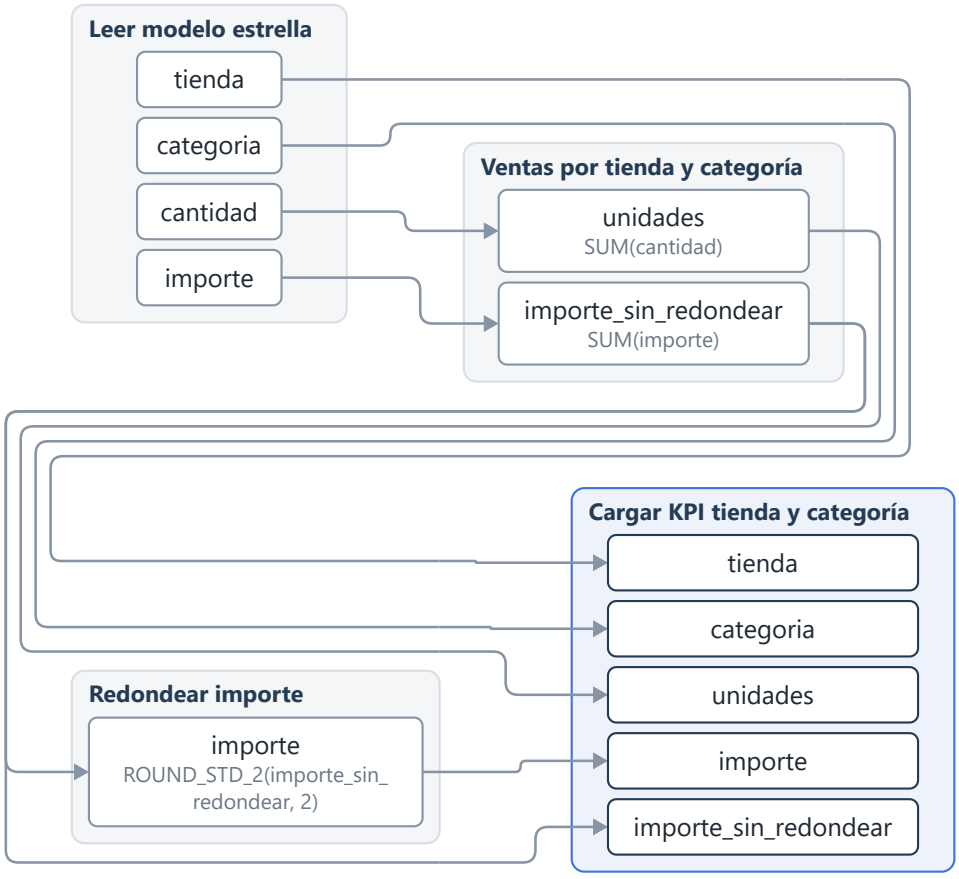
Campos de «CSV KPI tienda y categoría»

Campo	Tipo	Viene de	Transformaciones
tienda	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): tienda	sin cambios

Campo	Tipo	Viene de	Transformaciones
categoria	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): categoria	sin cambios
unidades	Integer	Leer modelo estrella (consulta SQL a dw_grano_de_oro): cantidad	SUM(cantidad) en «Ventas por tienda y categoría»
importe_sin_redondear	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Ventas por tienda y categoría»
importe	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Ventas por tienda y categoría» → ROUND_STD_2(importe_sin_redondear, 2) en «Redondear importe»

Motor: metaverse de Pentaho y motor propio de Pentaho Live; metaverse no pudo trazar importe, así que esos campos los traza el motor propio.

Linaje hacia «Cargar KPI tienda y categoría»



Campos de «Cargar KPI tienda y categoría»

Campo	Tipo	Viene de	Transformaciones
tienda	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): tienda	sin cambios
categoria	String	Leer modelo estrella (consulta SQL a dw_grano_de_oro): categoria	sin cambios

Campo	Tipo	Viene de	Transformaciones
unidades	Integer	Leer modelo estrella (consulta SQL a dw_grano_de_oro): cantidad	SUM(cantidad) en «Ventas por tienda y categoría»
importe_sin_redondear	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Ventas por tienda y categoría»
importe	Number	Leer modelo estrella (consulta SQL a dw_grano_de_oro): importe	SUM(importe) en «Ventas por tienda y categoría» → ROUND_STD_2(importe_sin_redondear, 2) en «Redondear importe»

Motor: metaverse de Pentaho y motor propio de Pentaho Live; metaverse no pudo trazar importe, así que esos campos los traza el motor propio.

6.4 Diccionario de datos

> 4 salidas finales con 16 campos en total.

Salida final: CSV KPI ticket medio

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
tienda	String	2147483647			Leer modelo estrella	tienda
importe_total	Number		2	0.00	Calcular ticket medio	ROUND_STD_2
num_tickets	Integer	10	0	0	Totales por tienda	
ticket_medio	Number		2	0.00	Calcular ticket medio	ROUND_STD_2

Nombres, orden, formato y largo como los escribe «CSV KPI ticket medio»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Salida final: Cargar KPI ticket medio

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
tienda	String	2147483647			Leer modelo estrella	tienda
importe_total	Number		2		Calcular ticket medio	ROUND_STD_2
num_tickets	Integer	10	0		Totales por tienda	
ticket_medio	Number		2		Calcular ticket medio	ROUND_STD_2

Nombres, orden, formato y largo como los escribe «Cargar KPI ticket medio»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Salida final: CSV KPI tienda y categoría

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
tienda	String	2147483647			Leer modelo estrella	tienda
categoria	String	2147483647			Leer modelo estrella	categoria
unidades	Integer		0	0	Ventas por tienda y categoría	
importe	Number		2	0.00	Redondear importe	ROUND_STD_2

Nombres, orden, formato y largo como los escribe «CSV KPI tienda y categoría»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Salida final: Cargar KPI tienda y categoría

Campo	Tipo	Largo	Precisión	Máscara	Origen	Comentario
tienda	String	2147483647			Leer modelo estrella	tienda
categoria	String	2147483647			Leer modelo estrella	categoria
unidades	Integer		0	####0;-####0	Ventas por tienda y categoría	
importe	Number		2		Redondear importe	ROUND_STD_2

Nombres, orden, formato y largo como los escribe «Cargar KPI tienda y categoría»; el tipo, el origen y el comentario vienen del flujo. Los largos vacíos significan «sin límite».

Anexo: campos de los demás steps

Leer modelo estrella

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
categoria	String	2147483647		Leer modelo estrella
id_ticket	String	2147483647		Leer modelo estrella
cantidad	Integer	9	0	Leer modelo estrella
importe	Number	0		Leer modelo estrella

Ordenar por tienda

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
categoria	String	2147483647		Leer modelo estrella
id_ticket	String	2147483647		Leer modelo estrella
cantidad	Integer	9	0	Leer modelo estrella
importe	Number	0		Leer modelo estrella

Ordenar por tienda y categoría

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
categoria	String	2147483647		Leer modelo estrella
id_ticket	String	2147483647		Leer modelo estrella
cantidad	Integer	9	0	Leer modelo estrella
importe	Number	0		Leer modelo estrella

Totales por tienda

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
importe_sin_redondear	Number			Totales por tienda
num_tickets	Integer	10	0	Totales por tienda

Calcular ticket medio

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
importe_sin_redondear	Number			Totales por tienda
num_tickets	Integer	10	0	Totales por tienda
importe_total	Number		2	Calcular ticket medio
ticket_medio	Number		2	Calcular ticket medio

Ventas por tienda y categoría

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
categoria	String	2147483647		Leer modelo estrella
unidades	Integer		0	Ventas por tienda y categoría
importe_sin_redondear	Number			Ventas por tienda y categoría

Redondear importe

Campo	Tipo	Largo	Precisión	Origen
tienda	String	2147483647		Leer modelo estrella
categoria	String	2147483647		Leer modelo estrella
unidades	Integer		0	Ventas por tienda y categoría
importe_sin_redondear	Number			Ventas por tienda y categoría
importe	Number		2	Redondear importe

6.5 Modelo destino

Escribe en 2 tablas (kpi_ticket_medio_tienda, kpi_ventas_tienda_categoria): 8 columnas en total. Las claves y los tipos salen de los steps, sin consultar la base.

kpi_ticket_medio_tienda	
BLOB	tienda
NUMERIC	importe_total
INTEGER	num_tickets
NUMERIC	ticket_medio

kpi_ventas_tienda_categoria	
BLOB	tienda
BLOB	categoria
NUMERIC	unidades
NUMERIC	importe

dw_grano_de_oro › kpi_ticket_medio_tienda — Cargar KPI ticket medio (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
tienda	tienda	String	BLOB	
importe_total	importe_total	Number	NUMERIC	
num_tickets	num_tickets	Integer	INTEGER	
ticket_medio	ticket_medio	Number	NUMERIC	

dw_grano_de_oro › kpi_ventas_tienda_categoria — Cargar KPI tienda y categoría (Salida a tabla, vacía la tabla)

Columna	Campo del stream	Tipo Kettle	Tipo SQL (generado)	Clave / uso
tienda	tienda	String	BLOB	
categoria	categoria	String	BLOB	
unidades	unidades	Integer	NUMERIC	
importe	importe	Number	NUMERIC	

Nota. No se consultó la base: no se muestran las claves primarias ni foráneas, y los tipos SQL son los que generaría Kettle. Con «Consultar la base» se leen.

6.6 Validación

Sin errores ni advertencias.

ERRORES	ADVERTENCIAS
0	0

6.7 Conexiones y parámetros

1 conexión y 0 parámetros.

Conexiones a bases de datos

Nombre	Tipo	Acceso	Host	Base	Usada por
dw_grano_de_oro	SQLite	Native		${Internal.Entry.Current.Directory}/../salida/grano_de_oro_dw.sqlite	Cargar KPI ticket medio, Cargar KPI tienda y categoría, Leer modelo estrella

Nunca se incluyen contraseñas.